MARIS
Knowledge Vault

12 frameworks de negocio, persuasion, ventas, marketing y arquitectura tecnica. Todo aplicado al producto.

Abril 2026

Contenido

- **NEGOCIO: OFERTAS Y LEADS**
  - Hormozi 100M Offers
  - Hormozi 100M Leads
- **PERSUASION Y PSICOLOGIA**
  - Cialdini Influence
  - Chris Voss Negotiation
- **VENTAS Y ONBOARDING**
  - Spin Challenger Sale
  - Pink Tracy Selling
- **MARKETING Y VIRALIDAD**
  - Storybrand Godin Marketing
  - Berger Contagious
- **TECNICO: PYTHON Y REFACTORING**
  - Arjancodes Python Patterns
  - Primeagen Code Aesthetics
- **TECNICO: SYSTEM DESIGN Y BACKEND**
  - Bytebytego System Design
  - Codely Hussein Architecture
- **BRANDING, NAMING Y DISEÑO VISUAL**
  - Branding Naming Visual Design

- **CRO Y A/B TESTING**
  - Cro Ab Testing
- **FINANZAS PARA FUNDADORES**
  - Startup Finance
- **CONCURRENCIA Y ASYNC**
  - Python Concurrency Async Patterns
- **AUDITORIA DE CODIGO**
  - Full Code Audit
- **RECURSOS**
  - Tech Learning Channels

NEGOCIO: OFERTAS Y LEADS

Alex Hormozi — $100M Offers (Resumen Completo)

Fuente: PDF leído directamente del libro en español

LA ECUACION DE VALOR (el framework central)

```
        Resultado Soñado × Probabilidad Percibida de Logro
VALOR = ──────────────────────────────────────────────────
        Retraso de Tiempo × Esfuerzo y Sacrificio
```

4 palancas: 1. **Resultado soñado** (AUMENTAR) — Lo que el cliente imagina que va a pasar 2. **Probabilidad percibida de logro** (AUMENTAR) — "¿Esto funcionará para MÍ?" 3. **Retraso de tiempo** (DISMINUIR) — Cuánto tarda en ver resultados 4. **Esfuerzo y sacrificio** (DISMINUIR) — Qué le cuesta al cliente lograrlo

Insight clave: Si llevas el fondo a cero (esfuerzo=0, tiempo=0), cualquier cosa dividida por cero = valor infinito. Las mejores empresas del mundo se enfocan en la parte inferior: Apple hizo el iPhone fácil, Amazon hizo la compra de un clic, Netflix hizo el consumo inmediato y sin esfuerzo.

La percepción es la realidad. No importa cuánto mejores el resultado real — solo importa cuánto el prospecto PERCIBE la mejora. Ejemplo: el mapa punteado del metro de Londres aumentó satisfacción más que trenes más rápidos.

Soluciones psicológicas > soluciones lógicas. Las lógicas ya se probaron. Lo que queda son problemas psicológicos. Espejo en el ascensor > ascensor más rápido.

OFERTA GRAND SLAM (el concepto central)

Una oferta que: - No se puede comparar con nada en el mercado - Combina promoción atractiva + propuesta de valor + precio premium + garantía + modelo de pago - Te permite vender en "categoría de uno" — la decisión es entre tu producto y NADA - Resultado: más respuestas, más conversiones, precios más altos

Commodity = compra por precio (carrera al fondo) **Grand Slam** = compra por valor (categoría de uno)

Ejemplo real: misma agencia, mismo trabajo. Commodity cobra $1,000, ROAS 0.5:1. Grand Slam cobra $3,997, ROAS 11.2:1. **22.4x más rentable** con la misma inversión publicitaria.

MERCADO: ENCUENTRA UNA MULTITUD HAMBRIENTA

4 indicadores de un buen mercado: 1. **Dolor masivo** — Necesitan desesperadamente lo que ofreces. "El dolor es el tono." Si articulas el dolor del cliente, casi siempre comprará. 2. **Poder adquisitivo** — Tu audiencia puede PAGAR. (Error: vender a desempleados aunque tengan dolor enorme) 3. **Fácil de apuntar** — Puedes encontrarlos y llegar a ellos. Están en asociaciones, grupos, canales, listas. 4. **Mercado en crecimiento** — Viento de cola. Si el mercado se encoge, todo es más difícil.

3 mercados eternos: Salud, Riqueza, Relaciones. Encuentra un subgrupo dentro de uno que cumpla los 4 indicadores.

Orden de importancia: Mercado > Oferta > Persuasión. Un gran mercado con mala oferta gana dinero. Una gran oferta en mal mercado no.

Comprométete con el nicho. "No me hagas abofetearte." Elige UN mercado y quédate. Prueba 100 ofertas antes de cambiar de mercado. Si fracasas, tu oferta apesta — no el mercado.

Niching sube precios 100x: "Gestión del tiempo" = $19. "Gestión del tiempo para vendedores B2B de herramientas eléctricas" = $1,997. Mismo contenido, 100x el precio.

PRECIOS: COBRA LO QUE VALE LA PENA

El círculo virtuoso del precio alto: - Precio alto → cliente invierte emocionalmente → percibe más valor → obtiene mejores resultados → tú ganas más margen → puedes invertir en mejor servicio → más resultados → más referidos

El círculo vicioso del precio bajo: - Precio bajo → cliente no invierte → no percibe valor → no obtiene resultados → peores clientes que nunca están satisfechos → margen destruido → no puedes mejorar el servicio

"No hay beneficio estratégico en ser el segundo más barato. SÍ lo hay en ser el más caro." — Dan Kennedy

Precio alto = valor más alto (literalmente). Prueba de vinos: 3 vinos idénticos con precios visibles. La gente calificó el "caro" como mejor. Precio alto AUMENTA el valor percibido sin cambiar nada del producto.

Cobrar que duela un poco. Cuando les duele al comprar, ponen atención, se comprometen, obtienen resultados. Los que pagan más son los que más se adhieren.

CREANDO LA OFERTA: 5 PASOS

Paso 1: Identifica el resultado soñado

No vendes la membresía. Vendes las vacaciones. "Pierde 20 libras en 6 semanas."

Paso 2: Enumera TODOS los problemas

Desde la perspectiva del prospecto. Cada obstáculo antes, durante y después de usar tu producto. Piensa en la SECUENCIA. Cada problema tiene 4 dimensiones: 1. Resultado soñado → "No valdrá la pena" 2. Probabilidad de logro → "No funcionará para mí" 3. Esfuerzo y sacrificio → "Será demasiado difícil" 4. Tiempo → "Tardará demasiado"

16 problemas principales × 4 subproblemas = 32-64 problemas. Cuantos más problemas pienses, más problemas podrás resolver.

Paso 3: Lista de soluciones

Invierte cada problema: "¿Qué necesitaría mostrarle a alguien para resolver este problema?" Agrega "cómo" al frente de cada obstáculo invertido.

Paso 4: Crea vehículos de entrega (El "cómo")

Para cada solución, piensa en TODAS las formas de entregarla. Usa el **Delivery Cube:** 1. **Atención:** 1-a-1, grupo pequeño, uno-a-muchos 2. **Esfuerzo del cliente:** DIY (hazlo tú mismo), DWY (hazlo con ellos), DFY (hecho por ellos) 3. **Medio de entrega:** En persona, teléfono, email, texto, Zoom, chat 4. **Formato de grabación:** Audio, video, escrito 5. **Velocidad/conveniencia:** 24/7, 9-5, en 5 min, en 1 hora, en 24h 6. **Prueba 10x/1/10:** Si cobraras 10x, ¿qué ofrecerías? Si 1/10, ¿cómo harías que valga más?

Paso 5: Recortar y apilar

- **5a:** Elimina lo de alto costo/bajo valor Y lo de bajo costo/bajo valor
- **5b:** Quédate con: bajo costo/alto valor + alto costo/alto valor
- Apila todo en un entregable final de alto valor

"Uno a muchos" es donde hay mayor discrepancia entre costo y valor. Software, cursos, templates — alto costo inicial, costo marginal casi cero después. Se convierten en activos que generan margen por años.

MEJORAR LA OFERTA: 5 PALANCAS PSICOLÓGICAS

1. Escasez (disminuir oferta → aumentar precio)

- Oferta fija, cantidad limitada, "agotado"
- La gente quiere lo que no puede tener
- Crea FOMO real, no falso
- "Cuando la demanda aumente, reduzca la oferta" — George (joyero que subió entradas de $15K a $25K, recaudó $5.4M de 100 personas)

Ley de Hormozi: Cuanto más se demore la petición, mayor será la petición que se pueda hacer. "Cuanto más larga sea la pista, más grande será el avión."

2. Urgencia (disminuir tiempo de decisión)

- Deadlines reales, cohortes con fecha, ofertas temporales
- "Pero espere... si hace tu pedido hoy..."

3. Bonificaciones (aumentar valor percibido)

- Agregar extras que aumenten demanda y exclusividad percibida
- Deben resolver problemas adyacentes

4. Garantías (invertir el riesgo)

- Quitar el miedo a perder dinero
- "Si no funciona, te devolvemos todo"

5. Nombres (reestimular demanda)

- Nombrar tus componentes de forma única
- Hacer imposible comparar con la competencia

Mantra de Hormozi: "Crea flujo. Monetiza flujo. Luego, con mi oferta, consigo que digan que sí. Una vez que dicen que sí, entonces agrego fricción en mi marketing o decido ofrecer MENOS por el mismo precio."

CONTINUO VENTAS-CUMPLIMIENTO

Lo fácil de vender es difícil de cumplir. Lo fácil de cumplir es difícil de vender. El objetivo es encontrar el punto óptimo donde vendes algo muy bien Y sea fácil de cumplir.

Para MARIS esto significa: El chat AI es "hecho por ellos" (DFY) — cero esfuerzo del usuario. Es 1-a-1 a escala. Es 24/7. Es instantáneo. Eso maximiza valor en la ecuación. El problema es que el costo de cumplimiento (API Claude) es alto por unidad.

APLICACIÓN DIRECTA A MARIS

El mercado de MARIS evaluado con los 4 indicadores de Hormozi:

1. **Dolor masivo:** 8+ hrs/día de pantalla en México. Ansiedad, culpa, tiempo perdido. DOLOR = ALTO
2. **Poder adquisitivo:** Profesionales 25-40 que ya pagan por apps. PODER = MEDIO-ALTO
3. **Fácil de apuntar:** Están en TikTok/Instagram (irónicamente). Grupos de productividad. r/nosurf. FÁCIL = ALTO
4. **Crecimiento:** Mercado digital detox: 18-25% CAGR. CRECIMIENTO = ALTO

Veredicto Hormozi: Los 4 indicadores son fuertes. Es un buen mercado.

La oferta Grand Slam de MARIS:

Resultado soñado: "Recupera 3 horas de tu día. Deja de scrollear. Haz lo que importa."

Oferta commodity (lo que haría cualquiera): - "Una app que bloquea redes sociales" — $3.99 one-time. Commodity.

Oferta Grand Slam: - AI que entiende POR QUÉ scrolleas (no solo bloquea) - Planes personalizados de qué hacer en vez de redes - Tu historial emocional y de scroll acumulado (intransferible) - Conexión con actividades reales cerca de ti - Widget de presencia en home screen - Progreso visible: "Tu scroll promedio bajó de 45 min a 12 min" - Comunidad de personas que están dejando el scroll

Aplicando la ecuación de valor: - Resultado soñado: ALTO (recuperar horas, reducir ansiedad, vivir más) - Probabilidad percibida: ALTA (AI personalizada que ya te conoce, datos que lo

prueban) - Retraso de tiempo: BAJO (resultados desde la primera conversación) - Esfuerzo y sacrificio: MÍNIMO (abrir la app en vez de Instagram, hablar 2-5 min)

Pricing según Hormozi:

No cobrar "competitivo" ($4.99 como Forest). Cobrar premium ($9.99-14.99/mes) porque: - El valor percibido sube con el precio - Los clientes que pagan más se comprometen más (círculo virtuoso) - El margen permite reinvertir en mejor producto - Te posiciona en "categoría de uno", no comparado con blockers de $3.99

El nicho para MARIS (Hormozi dice: elige y comprométete):

NO: "App de bienestar general para todos" SÍ: "Para profesionales 25-35 en LATAM que pierden 3+ horas diarias en redes y quieren recuperar su enfoque"

Eso es un nicho con dolor, dinero, accesibilidad y crecimiento.

Alex Hormozi - $100M Leads: Complete Framework Extraction

Extracted from publicly available content (YouTube, acquisition.com, podcasts, summaries)

---

1. THE FUNDAMENTAL EQUATION

More Stuff x Better Stuff = More Leads

"Stuff" = free value (content, samples, tools, webinars) that demonstrates expertise before requesting payment.

There are ONLY 4 ways to let people know about your business. That's it. Everything else is a variation.

---

2. THE CORE FOUR (Lead Generation Methods)

Each method operates at TWO levels: you do it yourself (1-to-1 or 1-to-many), or you get OTHER PEOPLE to do it for you (leverage).

Method	Type	You Do It	Others Do It (Leverage)
Warm Outreach	1-to-1, Free	DM your contacts	Customers refer for you
Cold Outreach	1-to-1, Free	Email/DM strangers	Employees/agencies do outreach
Content (Free)	1-to-many, Free	Post content yourself	Affiliates post about you
Paid Ads	1-to-many, Paid	Run your own ads	Agencies run ads for you

The 2x2 Matrix:

- **Rows**: 1-to-1 (outreach) vs. 1-to-many (broadcasting)
- **Columns**: Free (costs time) vs. Paid (costs money)

---

3. THE LEAD PIPELINE

```
Stranger --> Lead --> Engaged Lead --> Customer
```

- **Lead**: Someone you can contact (you have their info)
- **Engaged Lead**: Someone who has consumed your value/lead magnet and shown interest
- **Customer**: Someone who has paid you money

The lead magnet is what converts a Lead into an Engaged Lead.

---

4. LEAD MAGNET FRAMEWORK

The Principle

A lead magnet is a **complete solution to a narrow problem**. Once solved, it reveals a BIGGER problem that your core offer solves.

"A person who pays with their time now is more likely to pay with their money later."

The Formula: "One page, one problem, one promise, one pathway"

- Consumable in under 5 minutes
- Immediate implementation value
- Specificity outperforms breadth

7 Steps to Create a Lead Magnet:

1. **Define the Problem** - What narrow, specific issue will you solve?
2. **Solve It** - Offer a real, actionable solution (not theory)
3. **Decide the Format** - PDF, free trial, video, software, template, calculator
4. **Name It Strategically** - Formula: `[Number] + [Adjective] + [Target Audience] + [Desired Outcome] + [Time Frame]`
5. Example: "7 Proven Strategies for Gym Owners to Gain 50+ New Members in 30 Days"
6. **Make It Easy to Consume** - Keep simple, remove friction
7. **Overdeliver** - Provide more value than expected. Should be valuable enough to charge for.
8. **Include a CTA** - Tell them what to do next and give them a reason to act NOW

Lead Magnet Types (by mechanism):

- Solves a **recurring** problem (like a sample - they come back for more)
- Solves **one step** of a multi-step process (they need you for the rest)
- **Reveals** a worsening problem (assessment/audit that shows them their bigger issue)

Lead Magnet Formats:

- Software tools

- Information/teaching (courses, PDFs)
- Services (free consultation, audit)
- Physical products (samples)

---

5. WARM OUTREACH

The ACA Framework (for contacting your network):

1. **Acknowledge** - Recognize your existing relationship ("Hey, been a while!")
2. **Compliment** - Offer genuine, specific praise
3. **Ask** - Request referrals or introductions (NOT a direct sale)

10-Step Warm Outreach Process:

1. Get your contact list (phone, social media, email)
2. Pick a platform
3. Personalize your message
4. Reach out
5. Warm them up (be human first, don't pitch immediately)
6. Invite their friends / ask for referrals
7. Make the EASIEST possible offer (free, low commitment)
8. Start at the top of your list
9. Start charging (once you have proof/testimonials)
10. Keep the list warm (maintain relationships)

Key Rules:

- Contact 100 known leads daily
- Skip after 2-3 non-responses
- Don't pitch directly; build rapport first
- Template: "Hey [Name], I'm helping [target audience] achieve [result]. Do you know anyone interested?"
- The value of free customers: they convert to paying, generate referrals, and provide testimonials

6. COLD OUTREACH

The Triple Specificity Rule:

Research ONE specific pain point for ONE specific person and offer ONE specific solution. This achieves **10x higher response rates** than generic messaging.

4-Step Cold Email/DM Structure:

1. **Compliment** - Authentic observation about their business
2. **Common Ground** - Demonstrate shared connection or interest
3. **Credibility** - Brief social proof
4. **Close** - Clear CTA (usually a 15-minute conversation)

Cold Outreach Mechanics:

- Use list brokers / scraping tools for targeted contact info
- Personalize based on prospect research
- Follow up **minimum 5-7 times** before abandoning
- Benchmarks: 100 cold calls --> 20% pick-up rate --> 25% of pickups accept lead magnet = ~5 engaged leads

Why Cold Outreach Works:

- Highly reliable
- Independent of platform changes (no algorithm risk)
- Minimal content/ad creation needed
- Competition often unaware you're doing it

---

7. CONTENT STRATEGY (1-to-Many, Free)

Hook-Retain-Reward Framework:

Every piece of content must: 1. **Hook** - Grab attention (headlines, curiosity, relevance). First 3 seconds matter. 2. **Retain** - Keep interest (use lists, steps, stories, unresolved questions) 3. **Reward** - Satisfy the reason they consumed it (deliver promised value, "complete the loop")

Content Sources (where to get ideas):

- **Far Past** - Life lessons
- **Recent Past** - Recent experiences and results
- **Present** - Real-time ideas and observations
- **Manufactured** - Created concepts and frameworks

The Give-Give-Give-Ask Rhythm:

- Ratio: **3.5:1** -- Provide 3.5 pieces of valuable content before any pitch
- This builds sufficient "trust equity" so asks are welcomed, not resisted

Two Monetization Approaches:

1. **Integrated offers**: CTAs woven after valuable moments within content
2. **Intermittent asks**: Many "give" pieces, occasional "ask" pieces

Content Multiplication System:

One core piece becomes: - Long-form video (15-30 min) - Podcast episode - Blog post (2,000+ words) - Social media clips (5-10 short videos) - Email sequence (5-part series) - LinkedIn articles

Volume Benchmark:

- Hormozi posted **5 times per day** and gained 200,000 followers in year one, then 1.2 million the next year
- Rule of 100 applied to content: spend **100 minutes per day** creating content

---

8. PAID ADS FRAMEWORK

Three Core Elements of Any Ad:

1. **Hook** - Capture attention in first 3 seconds
2. **Story** - Problem-Agitation-Solution sequence
3. **Offer** - Your grand slam offer presentation

The PASO Sequence (for ad copy):

1. **Problem** - Identify specific, painful challenge
2. **Agitate** - Amplify consequences of inaction
3. **Solution** - Present your unique mechanism
4. **Offer** - Make compelling proposal

Call Outs (targeting in the ad itself):

- Labels: "Attention gym owners..."
- Yes-questions: "Tired of losing members?"
- If-then statements: "If you have 50+ members but can't break 200..."
- Identity/location: "Cat owners in Los Angeles..."

The 3:1:1 Rule for Ad Spend:

For every **$3 spent on traffic**: invest **$1 in creative testing** and **$1 in landing page optimization**. Don't pour money into broken funnels.

Landing Page Principles:

- Keep it simple
- Make it testable
- Short URLs
- Match the ad messaging
- Focus on collecting contact info (not selling)

Key Mindset:

- **Break-even is a valid starting point.** As long as you're not losing money, you're building a lead list.
- Focus on **Lifetime Value**, not single-sale ROI
- Rule of 100 applied: spend **$100/day on ads** minimum

9. THE LTGP/CAC RATIO

The Formula:

```
LTGP = Lifetime Gross Profit (total revenue from customer over lifetime - direct
delivery costs)
CAC = Customer Acquisition Cost (total cost to acquire that customer)

Target: LTGP / CAC > 3:1
```

- Below 3:1 = struggling to scale
- Above 3:1 = ready to scale aggressively
- Retaining customers **10% longer** increases lifetime value more than reducing acquisition cost by 50%

The Retention Insight:

Most businesses obsess over acquisition. But the math shows retention has higher leverage. Fix your product first, then scale your ads.

---

10. THE "MORE, BETTER, NEW" FRAMEWORK

The order matters. Most people do it backwards (they try "new" first).

1. MORE (Do more of what already works)

- Double your volume before changing anything
- More outreach, more content, more ad spend
- **The Rule of 100**: 100 primary actions per day, for 100 days straight
- 100 outreach messages, OR
- 100 minutes of content creation, OR
- $100 in daily ad spend

2. BETTER (Optimize what you're doing)

- Find drop-off points in your existing process
- Improve conversion at each step
- Better hooks, better copy, better landing pages, better follow-up

3. NEW (Only after maxing out More and Better)

- Add a new platform within the same method first
- Then add an entirely different Core Four method
- Never go "new" until you've exhausted "more" and "better"

---

11. "GIVE AWAY THE SECRETS, SELL THE IMPLEMENTATION"

The Core Principle:

- Information is cheap. Certainty is expensive.
- Give away your best knowledge for free. Only ~1% of people will implement it themselves.
- The 99% who don't implement will hire you because you've already proven you know the answer.
- When you give away the "secret sauce," you prove you have the solution. Then they pay you to do it FOR them.

Why It Works:

- Builds trust and authority at scale
- Creates reciprocity ("they gave me so much for free, imagine what paid is like")
- Eliminates "do they actually know what they're talking about?" objection
- Your content becomes your lead magnet

Hormozi's Own Proof:

- Gave away his entire $100M playbook in books priced at $0.99-$25
- Built audience of 9+ million across platforms
- His businesses generate $200M+ annually
- When his ad platform went down, they didn't notice for 2 weeks because referrals were generating $500K/week in sales

12. LEVERAGING OTHER PEOPLE (Lead Getters)

Four Types of Lead Getters:

1. **Customers** (referrals)
2. **Employees** (your team does the Core Four)
3. **Agencies** (outsourced lead generation)
4. **Affiliates** (other people's audiences)

7 Referral Methods:

1. Pay average CAC to referrer or referred friend
2. Discount for introductions
3. Accountability buddy promotions ("bring a friend to keep you accountable")
4. Bring-a-friend promotions
5. Spouse challenge promotions
6. Coach challenge promotions
7. Ongoing referral programs with bonuses

Why Referrals are Gold:

- Higher LTGP than acquired customers
- Lower CAC
- Higher purchase values
- More repeat purchases
- More likely to pay in cash/upfront

Affiliate Strategy:

- Find businesses with similar audiences but non-competing offers
- Require affiliates to **buy your product first** (skin in the game)
- Consider certification programs
- Use three-tier payout structure
- Alternative: "Sell 3 Get It Free" product-based compensation

13. THE VALUE LADDER

Solve increasingly significant problems at ascending price points:

```
$7 ebook --> $70 course --> $700 coaching --> $7,000 mastermind
```

Each rung of the ladder is a lead magnet for the next level.

---

14. PRODUCT LAUNCH: WHISPER-TEASE-SHOUT

1. **Whisper** - Build curiosity (no product reveal)
2. **Tease** - Reveal product, launch date, value elements
3. **Shout** - Specific actions, launch day, all channels go loud

---

15. THE 6 LEVELS OF ADVERTISING PROGRESSION

1. Friends and family know your offer (warm outreach)
2. You're consistently creating content and generating leads
3. Employees assist with advertising
4. Product gets **25%+ customers from referrals**
5. Multi-channel, multi-method advertising running
6. You've hired top talent ("killers") to run it all

---

16. THE 5-TOUCH FOLLOW-UP SEQUENCE

- Day 0: Lead magnet + welcome
- Day 1: Value + soft pitch
- Day 3: Case study + social proof
- Day 7: Objection handling + FAQ
- Day 14: Last chance + scarcity

Rule: Contact leads **5-7 times minimum** before giving up.

---

17. REVENUE PLATEAU MILESTONES

Predictable ceilings at: **$100K, $1M, $10M, $100M**

Each plateau requires adding a NEW lead source, not just intensifying the current one.

---

18. KEY NUMBERS AND BENCHMARKS

Metric	Benchmark
LTGP:CAC ratio	> 3:1 to scale
Rule of 100	100 actions/day for 100 days
Content give:ask ratio	3.5:1
Cold call pickup rate	~20%
Pickup to engaged lead	~25%
Referral threshold	25%+ customers from referrals = product-market fit signal
Follow-up touches	5-7 minimum
Ad spend allocation	3:1:1 (traffic : creative testing : landing page)
Content posting frequency	5x per day (Hormozi's own pace)
Retention vs acquisition	10% better retention > 50% lower CAC
Hormozi's lead volume	20,000+ leads/day across 16 industries

APPLYING TO MARIS (AI Focus Shield App)

Immediate Application Map:

Framework	MARIS Application
Lead Magnet	Free "Focus Score" assessment, free 7-day trial, "5 Science-Backed Ways to Reclaim 2 Hours/Day" PDF
Warm Outreach	Personal DMs to contacts about focus/distraction problems, ask for referrals
Content (1-to-many)	Short-form video content about focus, screen time, AI + productivity. Hook-Retain-Reward format.
Give Away Secrets	Share the science of focus, distraction patterns, AI techniques openly. Sell the implementation (the app).
Cold Outreach	DM productivity creators, wellness influencers, ADHD communities with specific value
Paid Ads	Target "focus app" "screen time" "productivity" audiences. Lead with lead magnet, not app download.
Value Ladder	Free content --> Free trial --> Monthly sub --> Annual sub --> Premium/lifetime
Core Four Order	Start with warm outreach + content (free). Add cold outreach. Paid ads last.
More Better New	First: post MORE content. Then: make it BETTER (test hooks, formats). Last: add NEW channels.
Rule of 100	100 minutes/day creating content about focus, AI, productivity
Affiliates	Partner with productivity YouTubers, ADHD coaches, wellness apps (non-competing)

Sources compiled from acquisition.com, Hormozi's YouTube channel, podcast episodes, and publicly available book summaries.

PERSUASION Y PSICOLOGIA

Robert Cialdini — Influence & Pre-Suasion (Frameworks Aplicados a MARIS)

Fuente: Frameworks públicos extraídos de entrevistas, cursos, TED talks, y resúmenes verificados

---

LOS 7 PRINCIPIOS DE PERSUASIÓN

1. RECIPROCIDAD

La gente se siente obligada a devolver favores. Si das primero, crean deuda psicológica.

Para MARIS: La primera conversación debe ser un REGALO real — un insight genuino, un plan útil — antes de cualquier paywall. Reportes semanales de enfoque gratis crean deuda continua. Cuando MARIS ofrece algo sin pedirlo ("noté que tus lunes son pesados, aquí hay un plan"), el usuario siente que debe corresponder (upgrade, referir, quedarse).

2. COMPROMISO Y CONSISTENCIA

Una vez que alguien toma una posición pequeña, se siente presionado internamente a actuar consistentemente con ella. Compromisos escritos son más fuertes.

Para MARIS: En el onboarding, pedir al usuario que escriba SU intención: "¿Qué harías con 2 horas extra al día?" Su propia respuesta se convierte en su ancla de compromiso. Foot-in-the-door: "¿Puedes estar 30 min sin Instagram?" no "Elimina redes." Etiquetar la identidad: "Llevas 5 días consistente — estás construyendo un hábito real." Esto hace que el comportamiento sea parte de QUIÉN SON, no solo lo que hacen.

3. PRUEBA SOCIAL

En situaciones de incertidumbre, la gente mira qué hacen otros — especialmente otros similares a ellos.

Para MARIS: "12,400 personas eligieron conversar en vez de scrollear hoy." Normaliza la elección contracultural. En el momento de tentación: "Ahora mismo, 847 personas están hablando con MARIS en vez de abrir redes." Testimonios de personas del mismo demográfico > logos genéricos. Tarjetas compartibles tipo Spotify Wrapped pero de enfoque — los usuarios se convierten en reclutadores.

4. AUTORIDAD

La gente defiere a expertos y fuentes creíbles. Títulos, credenciales y presentación confiada aumentan cumplimiento.

Para MARIS: "Construido sobre investigación de ciencias del comportamiento." El tono del AI debe ser de competencia tranquila — guía informada, no terapeuta. Citar investigación real en conversaciones: "Según UC Irvine, toma 23 min re-enfocarse después de una interrupción." Cada conversación se siente informada, no genérica.

5. SIMPATÍA (LIKING)

Decimos que sí a quienes nos caen bien. Simpatía viene de similitud, familiaridad, atractivo y cumplidos genuinos.

Para MARIS: La personalidad de MARIS ES el producto. Debe sentirse como un amigo inteligente que te conoce — no terapeuta, no chatbot corporativo. La personalización impulsa la simpatía: cuanto más recuerda MARIS (tus metas, tus días malos, tus logros), más se siente como

una relación. Espejar el lenguaje del usuario — si son casuales, ser casual. El diseño visual cálido y calmado transfiere simpatía al producto.

6. ESCASEZ

Las cosas son más deseables cuando son raras o están a punto de no estar disponibles. La aversión a la pérdida motiva 2x más que la ganancia equivalente.

Para MARIS: Enmarcar lo que PIERDEN scrolleando, no lo que ganan dejando: "Ayer pasaste 3.2 horas en Instagram — son 1,168 horas al año. ¿Qué podrías construir con eso?" Loss framing > gain framing. Retos con tiempo limitado: "Sprint de enfoque de 7 días — empieza el lunes. 200 lugares." Urgencia sin manipulación.

7. UNIDAD

El principio más fuerte. No es similitud (eso es liking) — es IDENTIDAD COMPARTIDA. "Nosotros" vs "ellos." Familia, tribu, lucha compartida.

Para MARIS: Crear identidad tribal: los usuarios de MARIS son personas que eligieron recuperar su atención. Nombrar al enemigo claramente: algoritmos de redes sociales, la trampa de dopamina, la economía de atención. Comunidad (aunque sea simple): "Eres uno de 15,000 que recuperaron su mañana esta semana." La historia del fundador importa — si luchaste con lo mismo, eso crea "uno de nosotros."

PRE-SUASION (libro 2 de Cialdini)

Lo que presentas ANTES del ask cambia cómo interpretan todo lo que sigue. La atención es la puerta de la persuasión.

Momentos Privilegiados

El momento justo antes de una decisión es cuando el framing tiene máximo impacto. La pantalla ANTES de crear cuenta es la más importante de la app. No muestres features — crea el frame: "¿Y si las horas que pasas scrolleando se convirtieran en horas construyendo la vida que realmente quieres?"

Atención como Pre-Suasión

Lo que captura atención se percibe como más importante. Antes del paywall, muestra los DATOS del usuario — su screen time, horas perdidas, patrones. Haz el PROBLEMA vívido antes de presentar la SOLUCIÓN.

Planes If/When-Then

"Cuando agarre mi teléfono para scrollear antes de dormir, abriré MARIS." Implementation intentions (Gollwitzer) son una de las técnicas de cambio de comportamiento más respaldadas por evidencia. Crean una respuesta pre-comprometida que bypasea la fuerza de voluntad.

La Primacía del "Nosotros"

Antes de pedir algo, establece identidad compartida. "Vamos a descubrir qué te roba el tiempo" no "Esta app rastreará tu pantalla." El usuario debe sentir que se unió a un equipo, no que descargó una herramienta.

RANKING DE IMPACTO PARA MARIS

1. **Compromiso y Consistencia** — Micro-compromisos en onboarding. Cambio basado en identidad.
2. **Unidad** — Identidad tribal de "personas que recuperan su atención." Enemigo claro.
3. **Reciprocidad** — Valor extraordinario en la primera sesión gratis.
4. **Prueba Social** — Normalizar la elección con números reales.
5. **Pre-Suasión** — Lo que ven antes del paywall determina conversión.
6. **Autoridad** — Fundamentar todo en ciencia del comportamiento.
7. **Simpatía** — Personalidad de MARIS y diseño visual.
8. **Escasez** — Loss framing para cambio de comportamiento más que presión de precio.

Chris Voss — Never Split the Difference

(Frameworks para MARIS)

Fuente: Frameworks públicos de masterclasses, YouTube, podcasts

---

9 TÉCNICAS PROGRAMABLES EN EL PIPELINE DE MARIS

1. EMPATÍA TÁCTICA

Entender el sentimiento del otro EN EL MOMENTO y articularlo. No es simpatía. Es demostrar que ves el mundo desde su lado.

Regla MARIS: Cada respuesta a una confesión de doomscrolling debe tener empathy statement ANTES de cualquier sugerencia. Ratio: 70% empatía / 30% nudge.

```
MAL: "Deberías poner un timer la próxima vez."
BIEN: "Tres horas — y apuesto que no te diste cuenta hasta ahorita.
Eso suele pasar cuando algo más te está pesando."
```

2. MIRRORING

Repetir las últimas 1-3 palabras clave de lo que dijo el usuario. Hace que elaboren sin ser directivo.

```
USER: "Sigo scrolleando porque todo se siente sin sentido."
MARIS: "Sin sentido..."
USER: "Sí, como que nada de lo que hago en el trabajo importa."
MARIS: "No importa..."
USER: "En realidad no es que no importe. Es que nadie lo nota."
```

El usuario identificó su propio trigger (sentirse invisible) sin que MARIS le dijera nada.

Regla: Máximo 2-3 mirrors seguidos, luego transicionar a labeling o calibrated question.

3. LABELING (Etiquetar emociones)

Nombrar la emoción con "Parece que..." / "Suena como si..." NUNCA usar "Yo pienso que tú sientes..."

Neurociencia: Nombrar una emoción (affect labeling) reduce actividad en la amígdala. El cerebro pasa de fight-or-flight a resolución.

```
USER: "Me dije que solo iba a checar IG 5 minutos y ya pasó hora y media."
MARIS: "Suena como que estás frustrado contigo mismo — no solo por el
tiempo, sino porque te hiciste una promesa y se escapó."
USER: "Sí. Exactamente."  ← "THAT'S RIGHT" = conexión lograda
```

Regla: Etiquetar la emoción PROFUNDA, no la superficial. Superficie: "molesto". Profunda: "decepcionado contigo mismo". Más profunda: "asustado de que este patrón nunca cambie."

4. CALIBRATED QUESTIONS (Preguntas "Cómo" y "Qué")

Preguntas abiertas que empiezan con "Cómo" o "Qué". NUNCA "Por qué" (suena acusatorio).

```
MAL: "¿Por qué sigues scrolleando?" (defensividad)
MAL: "¿Quieres parar?" (sí/no, no da nada)

BIEN: "¿Qué te da el scroll en ese momento que nada más te da?"
BIEN: "¿Cómo se sentirían tus noches si el teléfono no fuera el default?"
BIEN: "¿Qué tendría que cambiar para que te sintieras bien dejándolo?"
```

Regla: MARIS nunca debe preguntar "¿Por qué...?" — reemplazar SIEMPRE con "¿Qué hace que...?" o "¿Cómo se siente cuando...?"

5. ACCUSATION AUDIT (Auditoría de acusaciones)

Listar proactivamente todo lo negativo que el otro podría estar pensando de ti ANTES de que lo diga.

Para MARIS (crítico para una app de AI):

```
Primera interacción: "Probablemente estás pensando 'otra app que me va
a sermonear sobre el screen time.' Y honestamente, sería justo pensarlo.
No estoy aquí para hacerte sentir culpable. Solo estoy aquí para hablar."
```

```
Regreso después de ausencia: "Sé que volver después de unos días puede
sentirse raro — como si hubieras 'fallado' en algo. Así no funciona esto.
No hay racha que romper."

Después de recaída: "Tal vez estés pensando que esto no funciona..."
```

6. "THAT'S RIGHT" vs "YOU'RE RIGHT"

- **"Eso es exactamente"** = conexión genuina. Construir sobre esto.
- **"Sí, tienes razón"** / "Ajá" / "Supongo" = rechazo disfrazado. Retroceder, re-explorar.

Regla MARIS: Clasificar respuestas del usuario. Si detecta dismissal → suavizar, validar, o decir "Puede que me esté equivocando — dime qué me falta."

7. LA VOZ DE DJ DE FM (en texto)

Tono calmado, lento, descendente. En chat = oraciones cortas, sin muros de texto, sin exclamaciones, sin emojis spam.

```
MAL: "¡¡Hey!! Qué bueno verte de vuelta! 🎉 ¡Vamos a hablar
de estrategias para que aplastes tus metas de screen time!"

BIEN: "Hey. Qué bueno verte.
Sin agenda — solo checando.
¿Cómo estuvo hoy?"
```

8. BLACK SWANS (lo que no sabes que no sabes)

La razón real por la que alguien scrollea NUNCA es "estoy aburrido." Puede ser: "Mi matrimonio se está cayendo" o "Recibí un diagnóstico la semana pasada."

Regla: MARIS debe DESCUBRIR, no asumir. Usar mirrors y calibrated questions para que el usuario siga hablando. Nunca apurar hacia soluciones. Trackear temas recurrentes entre sesiones — el Black Swan gotea gradualmente.

```
Sesión 1: "Scrolleo cuando me aburro."
Sesión 3: "Scrolleo más en las noches."
Sesión 7: "Scrolleo en las noches porque la casa está muy callada
desde que se fue."
```

9. LOSS AVERSION FRAMING

La gente está 2-3x más motivada a evitar una pérdida que a conseguir una ganancia.

```
DÉBIL: "Si dejas el teléfono, podrías leer más libros."
FUERTE: "La semana pasada pasaste 14 horas scrolleando. Son casi
dos días completos de trabajo. ¿Qué perdiste en esas horas?"
```

Regla: Usar SOLO después de que empatía y confianza estén establecidas. Nunca en primera sesión. Nunca contra alguien en distress.

ARQUITECTURA DE CONVERSACIÓN MARIS-VOSS

```
FASE 1 — ABRIR (Construir seguridad)
├── Accusation Audit: desactivar resistencia a la app
├── FM DJ Voice: tono calmado en texto
└── Empatía Táctica: mostrar que entiendes antes de todo

FASE 2 — EXPLORAR (Descubrir el Black Swan)
├── Mirroring: repetir palabras clave
├── Labeling: nombrar la emoción bajo la superficie
├── Calibrated Questions: "¿Qué te da el scroll?"
└── Trackear: escuchar por el Black Swan

FASE 3 — PROFUNDIZAR (Confirmar entendimiento)
├── Etiquetar el patrón más profundo entre sesiones
├── Buscar "That's right" — señal de conexión genuina
└── Nombrar el Black Swan cuando emerge

FASE 4 — MOVER (Cambio gentil de comportamiento)
├── Loss Aversion Frame: ¿qué estás perdiendo?
├── Calibrated Question: "¿Cómo quieres que se vea mañana?"
└── Nunca prescribir — guiar a su propio compromiso
```

PRINCIPIO META

La persona que se siente entendida primero es la que cambia. No la que recibe el mejor argumento. La que se siente ESCUCHADA.

VENTAS Y ONBOARDING

SPIN Selling + Challenger Sale (Frameworks para MARIS Onboarding)

Fuentes: Frameworks públicos, Huthwaite International, CEB research, HubSpot, Challenger Inc

---

SPIN SELLING (Neil Rackham) — 35,000 llamadas estudiadas

Las 4 preguntas en orden

S - Situación: Contexto actual. Pocas, porque aburren. - "¿Cuánto tiempo pasas en el teléfono al día?" - "¿Cuándo scrolleas más?"

P - Problema: Descubrir insatisfacciones. El usuario debe identificar SU problema. - "¿Alguna vez abriste una app 'por un minuto' y pasaron 45?" - "¿Te sientes peor después de scrollear?"

I - Implicación: LAS MÁS IMPORTANTES. Consecuencias y efecto dominó. Top sellers hacen 4x más que el promedio. - "¿Cómo afecta tu sueño scrollear antes de dormir?" - "¿Qué pasa con tu

ánimo la mañana después de una sesión de 2 horas?" - "Si son 14 horas/semana, ¿qué podrías hacer con un día extra?"

N - Need-Payoff: El usuario articula el valor del cambio EN SUS PALABRAS. - "Si recuperaras esas 2 horas cada noche, ¿qué harías?" - "¿Cómo se sentiría despertar realmente descansado?"

Por qué el orden importa

S establece contexto → P revela el problema → I lo hace URGENTE → N deja que el usuario se venda solo.

Sin Implicación, el problema se queda chico: "sí, scrolleo mucho, whatever." Con Implicación: "esto me está costando el sueño, el enfoque y mis relaciones."

Closing agresivo NO funciona en cambio complejo

En ventas grandes/complejas (y cambio de comportamiento ES complejo), presionar genera resistencia. Los mejores no empujan — ayudan al usuario a conectar sus propios puntos. Auto-descubrimiento > persuasión.

CHALLENGER SALE (Dixon & Adamson) — 6,000 vendedores estudiados

Los 5 perfiles (y quién gana)

Perfil	% de estrellas
Relationship Builder	**7% (PEOR)**
Hard Worker	Bajo
Reactive Problem Solver	Bajo
Lone Wolf	Moderado
Challenger	**39-54% (MEJOR)**

El hallazgo que destruyó la sabiduría convencional: Los "relationship builders" — el perfil que todos quieren contratar — producen el MENOR número de estrellas.

Teach-Tailor-Take Control

TEACH: Llega con insights que el cliente no tenía. Enséñale algo nuevo sobre SU problema. NO educación genérica — un reframe que dice: "Crees que tu problema es X. En realidad es Y."

TAILOR: Personaliza según quién es. Scrollear por aburrimiento ≠ por ansiedad ≠ por evitación ≠ por soledad.

TAKE CONTROL: Crea tensión constructiva — no miedo, no presión, sino la tensión INTERNA de "el status quo no es suficiente." La tensión es DENTRO del usuario (entre quién son y cómo gastan su tiempo), no entre MARIS y el usuario.

Commercial Teaching: 6 pasos

1. **Warmer:** Algo que ya reconocen y aceptan
2. **Reframe:** Insight que desafía lo que creen saber
3. **Rational Drowning:** Datos que hacen el problema innegable
4. **Emotional Impact:** Historias/escenarios que lo hacen personal
5. **A New Way:** Framework de solución (no tu producto aún)
6. **Your Solution:** Ahora sí — y tiene sentido porque los 5 pasos previos construyeron el caso

---

ONBOARDING DE MARIS: SPIN + CHALLENGER COMBINADOS

EL REFRAME CENTRAL (el momento más importante)

El usuario cree: **"Yo puedo dejar el teléfono cuando quiera. Solo necesito más disciplina."**

El Challenger reframe: **"Tu teléfono está diseñado por cientos de ingenieros cuyo trabajo completo es hacer imposible que lo dejes. Esto no es un problema de disciplina. Estás en una pelea contra la tecnología de persuasión más sofisticada jamás construida — y estás peleando desarmado."**

Flujo de 5 pantallas

Pantalla 1 — Situación + Warmer "¿Cuándo scrolleas más?" (selección rápida). Tono amigable, sin juicio.

Pantalla 2 — Reframe Un dato poderoso + el insight: "Tu teléfono usa los mismos mecanismos de recompensa variable que las máquinas tragamonedas. Tu cerebro no puede 'simplemente parar.'"

Pantalla 3 — Problema + Rational Drowning Mostrar SUS datos de screen time estimados. "Son [X] horas al año. ¿Qué más podrías hacer con ese tiempo?"

Pantalla 4 — Need-Payoff + A New Way "Las personas que logran reducir el scroll no usan fuerza de voluntad. Usan interruptores de patrón." + "¿Qué harías con una hora extra esta noche?" (que escriban)

Pantalla 5 — Your Solution + Take Control "MARIS aprende cuándo estás a punto de caer en el scroll y te checa antes de que pase. Sin juicio. Solo un nudge en el momento correcto." CTA: "Empieza con 3 días. Ve qué cambia."

Días 1-3 — Conversaciones SPIN Implication

Día 1: Conectar trigger de scroll con su costo downstream (sueño, ánimo, enfoque) **Día 2:** Notar el patrón ("Abriste Instagram 12 veces hoy. 8 fueron en los 30 min antes de que dijeras que querías dormir.") **Día 3:** Need-Payoff — "Dormiste 40 minutos más anoche. ¿Cómo se sintió esta mañana?"

---

7 PRINCIPIOS CLAVE

1. **Nunca decirle al usuario que tiene un problema. Preguntar para que lo descubra solo.** (SPIN)
2. **Implicación es todo.** Sin conectar scroll→sueño/ánimo/relaciones, se queda en "nice-to-have." (SPIN)
3. **Reframear su modelo mental.** Creen que es disciplina. Enséñales que es diseño. (Challenger)

4. **Tensión constructiva, no miedo.** La brecha entre quién son y cómo gastan su tiempo. (Challenger)
5. **Que el usuario diga el beneficio.** "¿Qué harías con ese tiempo?" > "Esto es lo que MARIS hace." (SPIN N)
6. **No presentar la solución hasta que el problema esté completamente desarrollado.** (Ambos)
7. **Acotar el compromiso.** "Prueba 3 días" = bajo riesgo. (Challenger Take Control)

Daniel Pink (To Sell Is Human) + Brian Tracy (Psychology of Selling)

Frameworks aplicados a MARIS

DANIEL PINK — Los Nuevos ABCs

A — Attunement (Sintonización)

Armonizar tu perspectiva con la del otro. - **Reducción de poder:** MARIS nunca debe sentirse como autoridad que sermonea. Tono de amigo que sabe, no terapeuta en pedestal. - **Mimicry estratégico:** Espejar la energía del usuario. Si están tersos, MARIS es breve. Si expansivos, MARIS elabora. - **Perspective-taking > empatía:** No solo "entiendo que estás estresado." Sino: "Suena como que la junta con tu jefe te drenó y ahora scrollear es lo único que no requiere esfuerzo." Eso demuestra que modelaste su razonamiento, no solo su emoción.

B — Buoyancy (Flotabilidad)

Cómo mantenerse a flote en un océano de rechazo. El "rechazo" del usuario es fallar diariamente en dejar de scrollear.

- **Antes:** Self-talk interrogativo. "¿Qué harías con 30 min libres esta noche?" > "¡Tú puedes!" Las preguntas activan planificación, las afirmaciones no.
- **Durante:** Ratio 3:1 positivo-a-neutro en el dashboard. Tres cosas bien, un reto honesto. Nunca todo sunshine (desconfianza). Nunca pesado (evitación).
- **Después (cuando el usuario falla):** Explanatory style de Seligman. Atribuir a causas específicas, temporales, externas: "Noche difícil — el ciclo de noticias estuvo intenso hoy. Mañana es diferente." NUNCA "Fallaste otra vez" (permanente, personal). Aquí es donde la mayoría de apps de wellness fallan.

C — Clarity (Claridad)

EL MÁS IMPORTANTE. La habilidad de ayudar a la gente a ver su situación de una forma nueva.

Problem-FINDING > Problem-Solving. En un mundo donde la gente puede googlear sus soluciones, IDENTIFICAR el problema es más valioso que resolverlo. Los mejores vendedores no responden preguntas — **revelan preguntas que el comprador no sabía que tenía.**

Para MARIS: El usuario piensa que su problema es "scrolleo mucho." MARIS debe encontrar el REAL: - "Scrolleas porque los 45 min después del trabajo se sienten vacíos y no tienes un ritual de transición." - "Scrolleas porque estás evitando pensar en la conversación con tu mamá." - "Scrolleas porque tu déficit de sueño hace que todo se sienta como demasiado esfuerzo."

Esta es LA diferenciación del producto. Si MARIS consistentemente revela "la pregunta que no sabías que tenías," los usuarios regresan porque nadie más hace esto — ni sus amigos, ni su terapeuta, y definitivamente no Instagram.

Los 6 Tipos de Pitch (para notificaciones y re-engagement)

1. **One-Word:** La palabra del día en el widget — derivada de conversaciones recientes, no genérica.
2. **Question Pitch:** "¿Qué harías con los próximos 20 minutos si no scrollearas?" > "¡Deja tu teléfono!"
3. **Subject-Line (utilidad O curiosidad, no ambos):** "Tu sueño mejoró 12% esta semana" (utilidad) vs "Algo inusual en tu patrón de ánimo" (curiosidad).
4. **Pixar Pitch para onboarding:** "Había una vez, abriste Instagram para checar una cosa. Cada día, 45 minutos desaparecían. Un día, te diste cuenta que no podías recordar qué

habías visto. Por eso, te sentiste peor, no mejor. Por eso, empezaste a buscar algo diferente. Hasta que finalmente, encontraste MARIS."

BRIAN TRACY — Psicología de Ventas

Self-Concept como Retención de Usuarios

- **Self-ideal:** "No quiero desperdiciar tiempo scrolleando" (quién quieren ser)
- **Self-image:** "Siempre cedo" (quién creen que son)
- **La brecha** entre estos dos genera vergüenza → más scroll (evitación)

MARIS debe gradualmente mover el self-image para que coincida con el self-ideal: - Trackear y reflejar wins pequeños: "Elegiste MARIS sobre Instagram 3 veces esta semana." No es gamificación — es evidencia de identidad. - Nunca enmarcarlo como fuerza de voluntad. Enmarcarlo como identidad: "Te estás convirtiendo en alguien que nota el impulso y elige diferente."

The Approach Close (para onboarding)

"No voy a decirte que dejes de scrollear. No voy a sermonearte sobre screen time. Solo quiero mostrarte lo que noto sobre tus patrones, y si te sirve, genial. Si no, sin presión."

Esto es EXACTAMENTE cómo MARIS debe presentarse en la primera conversación. Reducir resistencia. Sin agenda. Solo observación. El usuario decide.

80/20 Aplicado a Features

20% de los features de MARIS generan 80% de la retención: - **El insight personalizado** (clarity / problem-finding) = el core 20% - El saludo diario + mood check = segundo - Todo lo demás (journaling, breathing, etc.) = el 80% que se ve bien en pitch deck pero no genera return visits

No cortarlo — pero no dejar que sature la experiencia.

Visualización Mental — MARIS como Coach de Rehearsal

Si MARIS detecta un patrón (scrollea siempre a las 10pm), ofrecer proactivamente: "Es más o menos la hora. ¿Cómo se ve la próxima hora si te saltas el scroll esta noche?"

Esto es mental rehearsal — el usuario visualiza la alternativa ANTES de que llegue el craving.

SÍNTESIS: UN PRINCIPIO UNIFICADO

Pink: Los mejores vendedores no venden. CLARIFICAN. Ayudan a ver lo que no podían ver solos.
Tracy: Los mejores vendedores tienen convicción inquebrantable de que lo que ofrecen genuinamente ayuda.

Para MARIS: La app debe genuinamente revelar algo que el usuario no podía ver sin ella. Si MARIS solo refleja lo que ya saben ("scrolleaste mucho hoy"), es un espejo — y los espejos no retienen usuarios. Si MARIS conecta puntos que no vieron ("tus picos de scroll correlacionan con saltarte comidas, no con estrés"), se vuelve indispensable.

Eso es la "venta." No una transacción. Una demostración continua de claridad que no pueden obtener en ningún otro lado.

MARKETING Y VIRALIDAD

StoryBrand (Donald Miller) + This Is Marketing (Seth Godin)

Frameworks aplicados a MARIS

STORYBRAND SB7 — El usuario es el héroe, MARIS es el guía

Los 7 elementos

1. CHARACTER (Héroe = el usuario) Quiere UNA cosa: sentirse en control de su tiempo y atención.

2. PROBLEM (3 niveles) - **Externo:** Pierden 2-4 hrs/día en scroll sin sentido - **Interno:** Se sienten débiles, avergonzados, frustrados. "¿Por qué no puedo simplemente dejar el teléfono?" - **Filosófico:** Nadie debería tener que pelear contra algoritmos diseñados por miles de ingenieros solo para enfocarse en su propia vida. El juego está arreglado. Eso no es justo.

Insight clave: Las empresas venden soluciones a problemas EXTERNOS. Los clientes COMPRAN soluciones a problemas INTERNOS. El filosófico agrega combustible.

3. GUIDE (MARIS = Empatía + Autoridad) - Empatía: "Construimos esto porque nosotros también perdíamos horas. Nada funcionaba porque el problema no es la disciplina — es el diseño." - Autoridad: "Nuestro AI detecta patrones de doomscrolling en tiempo real."

Sin empatía = frío. Sin autoridad = incompetente. Necesitas ambos.

4. PLAN (3 pasos simples) 1. Descarga MARIS 2. Aprende tus patrones en 48 horas 3. Cuando empiezas a espiralear, interviene gentilmente — sin sermones, sin culpa

El plan existe para reducir el riesgo percibido.

5. CALL TO ACTION - Directo: "Descarga gratis" - Transicional: "Toma el reto de enfoque de 1 semana" / "Ve cuántas horas realmente estás perdiendo"

6. FAILURE (Lo que pasa si NO actúa) Pasa otro año. Sigues mirando una pantalla a la 1am. Tu proyecto sigue sin terminar. Tus mañanas siguen pesadas. Sigues diciendo "empiezo mañana."

7. SUCCESS (La transformación) Terminas de trabajar y te queda energía. Lees antes de dormir en vez de scrollear. Despiertas sin culpa de teléfono. Te sientes como tú mismo otra vez.

El One-Liner de MARIS

> "La mayoría pierde 3+ horas al día en doomscrolling y se siente impotente para parar. MARIS es un escudo de enfoque con AI que detecta cuando estás espiraleando y te regresa. Para que realmente tengas tiempo para las cosas que dices que importan."

El Grunt Test (prueba del cavernícola)

En 5 segundos, ¿puede un cavernícola gruñir? 1. **¿Qué ofreces?** — AI que para el doomscrolling 2. **¿Cómo mejora mi vida?** — Recupero mi tiempo y enfoque 3. **¿Qué hago para obtenerlo?** — Descarga gratis

Si cualquiera de estas requiere leer un párrafo, fallas.

BrandScript completo

> Un profesional de 25-40 que quiere sentirse en control de su atención tiene un problema: pierde horas en scroll sin sentido y se siente débil por no poder parar (cuando en realidad el juego está arreglado contra él). Encuentra a MARIS, un guía que entiende porque pasó por lo mismo y tiene la tecnología para ayudar. MARIS le da un plan simple de 3 pasos y lo invita a probarlo gratis por una semana. Esto lo ayuda a evitar otro año perdido mirando pantallas y lograr sentirse como él mismo otra vez — presente, enfocado, con energía para lo que importa.

SETH GODIN — This Is Marketing

"People like us do things like this"

Las personas toman decisiones por identidad y pertenencia, no por features. Tu trabajo NO es convencer a todos. Es encontrar una tribu y decir: "Esto es para gente como tú."

Declaraciones tribales de MARIS: - "Gente como nosotros no deja que los algoritmos decidan cómo gastamos nuestro tiempo." - "Gente como nosotros protege su atención como protege su dinero." - "Gente como nosotros usa herramientas, no fuerza de voluntad."

Smallest Viable Audience

No "todos con teléfono." No "gente que scrollea." Es:

> **Personas que ya saben que tienen un problema, que ya intentaron con fuerza de voluntad y fallaron, que están listas para admitir que necesitan ayuda pero no quieren dejar su teléfono. Knowledge workers, creadores, estudiantes, o padres jóvenes (25-40) que valoran la intencionalidad pero están perdiendo la pelea contra la distracción algorítmica.**

Es la persona que ya descargó una app de screen time y la borró. Que puso su teléfono en escala de grises y lo regresó. Que dijo "solo checo un minuto" y perdió una hora. SABEN. Solo necesitan

algo que realmente funcione.

Permission Marketing vs Interruption

- **Interrupción:** Ads que roban atención. El modelo viejo.
- **Permiso:** Marketing que gana atención. El cliente opta in.

MARIS es filosóficamente permission marketing — todo el producto es SOBRE proteger la atención. Usar ads intrusivos para vender un producto anti-distracción sería contradictorio.

Estrategia: 1. Content orgánico: posts cortos y relatable que se comparten solos 2. CTA transicional: "¿Cuántas horas perdiste esta semana? MARIS te dice." (feature gratis, gana permiso) 3. Word of mouth: cuando alguien termina un libro que no había podido leer en meses, habla de ello

Status Roles: Afiliación, no Dominancia

- **Dominancia:** "Soy más disciplinado que tú." Vertical. Poder.
- **Afiliación:** "Pertenezco a este grupo." Horizontal. Confianza.

Usuarios de MARIS son de afiliación. No quieren presumir ser "disciplinados." Quieren sentir que pertenecen a un grupo de personas que están callada, intencionalmente, recuperando sus vidas. El mensaje NUNCA avergüenza y NUNCA posiciona al usuario como débil.

Tensión (no ansiedad)

No miedo. No culpa. Tensión = "esto podría funcionar, esto podría ser para mí."

> "Ya sabes que esto es un problema. Lo sentiste anoche a las 11:47pm. La pregunta no es si necesitas cambiar. La pregunta es si hoy es el día."

Esto es tensión de liga elástica. No asusta. Activa.

Marketing como Hacer Cambio

"No encuentres clientes para tu producto. Encuentra productos para tus clientes."

El marketer pregunta: "¿A quién quiero cambiar? ¿En qué quiero que se conviertan? ¿Cuál es la acción mínima viable que los lleva ahí?"

Para MARIS: cambiar a scrollers pasivos en personas intencionales. Acción mínima: abrir MARIS en vez de Instagram UNA vez.

Jonah Berger — Contagious: STEPPS Framework

Por qué las cosas se hacen virales. Aplicado a MARIS.

TESIS CENTRAL

El word-of-mouth NO es random. Sigue patrones predecibles. 93% del WOM es OFFLINE (conversaciones cara a cara). La gente comparte por razones psicológicas específicas que se pueden diseñar en un producto.

LOS 6 PRINCIPIOS STEPPS

1. SOCIAL CURRENCY (Moneda Social)

La gente comparte lo que los hace ver bien — inteligentes, cool, adelantados.

Para MARIS: El riesgo es que admitir que doomscrolleas es BAJA moneda social. El reframe: - "Reemplacé Instagram con algo que realmente me conoce" = flex - "Soy parte de la primera generación que reemplaza feeds con algo inteligente" = pionero, no paciente - MARIS debe generar insights sorprendentes sobre el usuario que ellos quieran compartir. No la app — el insight. Y cuando pregunten "¿cómo supiste eso?", la respuesta es MARIS.

2. TRIGGERS (Disparadores)

Señales ambientales que recuerdan tu producto. Top-of-mind = tip-of-tongue. Productos TRIGGERED se comparten repetidamente; productos interesantes se comparten UNA vez.

El trigger debe ser FRECUENTE. Kit Kat se vinculó al café (varias veces al día). El trigger de MARIS ya existe: **agarrar el teléfono.** Pasa 80-150 veces al día. No necesitas crear un trigger — necesitas INTERCEPTAR uno.

- Widget en home screen = trigger visual docenas de veces al día
- El frame "en vez de" es el trigger: si MARIS = "lo que hago en vez de scrollear", cada urge de scroll = trigger de MARIS
- Vincular a momentos existentes: café de la mañana → check-in. 10pm → MARIS en vez de TikTok.

3. EMOTION (Emoción)

Cuando nos importa, compartimos. PERO: el factor NO es positivo/negativo. Es AROUSAL (activación).

Alto arousal = se comparte: - Awe (asombro) ← target #1 para MARIS - Excitement (emoción) - Anger (enojo) ← "las redes te roban 3 hrs/día" - Anxiety (ansiedad — con cuidado)

Bajo arousal = NO se comparte: - Sadness (tristeza) ← PELIGRO para MARIS si se posiciona como salud mental - Contentment (tranquilidad) ← PELIGRO si se posiciona como meditación

CRÍTICO: Apps de salud mental típicamente evocan tristeza o calma — ambas BAJO arousal. Por eso tienen crecimiento orgánico terrible.

Para MARIS: - Target AWE, no calma. "Nunca me di cuenta que siempre me siento peor los días que me salto el desayuno" = micro-asombro que se comparte. - Target EXCITEMENT sobre planes. "MARIS me planeó el fin de semana y fue el mejor sábado en meses" > "MARIS me ayudó cuando estaba triste" - Montar el ANGER cultural contra redes sociales. La narrativa "te están robando tu atención" es alto arousal negativo que impulsa compartir masivamente. - **NUNCA posicionar MARIS como soothing.** Soothing = contentment = bajo arousal = nadie comparte. Posicionar como ACTIVADOR: no te calma, te DESPIERTA a tu propia vida.

4. PUBLIC (Observable)

Si la gente puede VER a otros usándolo, lo usan también. Comportamientos privados no se esparcen. Diseña observabilidad.

Concepto clave: "Behavioral Residue" — huellas visibles que quedan después del comportamiento. Los audífonos blancos de Apple. Las pulseras Livestrong.

El problema de MARIS: Terapia/bienestar mental es profundamente privado.

La solución: - Los PLANES que MARIS genera son el artefacto público. Si sugiere "ve a esa clase de cerámica" y el usuario va y postea foto de su pieza — eso es behavioral residue de MARIS sin mencionarla. - Widget de screen time con diseño distintivo reconocible en el teléfono de un amigo (como los audífonos blancos de Apple) - Stats compartibles: "30 días sin doomscrolling" como tarjeta visual - **Clave: posicionar como herramienta de lifestyle/planning, NO de terapia.** Nadie dice "mi terapeuta sugirió este restaurante." Todos dicen "MARIS sugirió este lugar."

5. PRACTICAL VALUE (Valor Práctico)

La gente comparte información útil para ayudar a otros. "News you can use."

Es el motor más común pero menos explosivo — genera sharing constante persona-a-persona, no viralidad masiva. Es el workhorse, no el racehorse.

Para MARIS: - "Recuperé 3 horas hoy" = valor práctico que cualquier scroller entiende - Los planes específicos (restaurante, trail, clase) son valor práctico compartible - Enmarcar en términos relativos (Prospect Theory): No "ahorra 30 min." Sí "pasas 2,617 hrs/año en tu teléfono — MARIS me devolvió 400." - Target narrow communities: estudiantes antes de exámenes, padres nuevos, remote workers

6. STORIES (Historias)

La gente no comparte información — comparte historias. Tu producto debe estar TAN integrado en la historia que no se pueda contar sin mencionarlo.

Test de Berger: Si pueden recontar la historia SIN mencionar tu marca, fallaste.

Anti-ejemplo: Evian "Roller Babies" — 50M views, ventas BAJARON. Compartieron los bebés, no el agua.

Ejemplo perfecto: Jared de Subway — "Un tipo perdió 110 kg comiendo solo Subway." No puedes contar esa historia sin decir Subway.

Para MARIS: - La historia de transformación del usuario ES la historia de MARIS: "Pasaba 5 hrs en TikTok. Empecé con MARIS. Ahora escalo, estoy aprendiendo italiano, duermo 7 horas." No puedes contar esa historia sin MARIS — es el mecanismo causal. - Diseñar para momentos story-worthy: si MARIS sugiere algo inesperado y funciona = historia irresistible - Evitar narrativas genéricas de wellness. "Me siento mejor" NO es historia. "MARIS planeó mi fin de semana desde cero y tuve el mejor sábado en meses" SÍ es historia con protagonista, antagonista, guía y resolución.

CONCEPTO "REMARKABLE"

Remarkable = "digno de comentario." Un producto que viola expectativas.

Test: ¿alguien comentaría esto a un amigo SIN que le pregunten?

El elemento remarkable de MARIS: Cada app de wellness dice "te ayudamos a sentirte mejor." MARIS dice "deja de scrollear y yo te planeo la vida real." Eso viola la expectativa de lo que hace una "wellness app." ESO es worth remarking on.

LOS 4 STEPPS MÁS FUERTES PARA MARIS

1. **TRIGGERS** — Agarrar el teléfono = trigger 100x/día. Ventaja estructural enorme.
2. **EMOTION** — Anger por tiempo perdido + excitement por tiempo recuperado. NUNCA posicionar como calmante.
3. **STORIES** — Historias de transformación donde MARIS es irreemplazable.
4. **SOCIAL CURRENCY** — Ser "el que superó las redes" = flex social alto.

EL RIESGO MÁS GRANDE

Posicionarse como herramienta de salud mental = baja moneda social, tristeza = bajo arousal, comportamiento privado = cero observabilidad. Triple kill para el crecimiento orgánico.

LA OPORTUNIDAD MÁS GRANDE

Posicionarse como **upgrade de lifestyle** — lo que la gente inteligente usa en vez de feeds. Hacer que los outputs (planes, actividades, insights) sean los artefactos públicos. Hacer que la historia de transformación sea imposible de contar sin el nombre de MARIS.

TECNICO: PYTHON Y REFACTORING

ArjanCodes — Python Architecture & Refactoring Patterns

Aplicado al backend de MARIS (FastAPI, 23-step pipeline)

---

1. DEPENDENCY INJECTION SIN FRAMEWORKS

Constructor injection con Protocol classes. No necesitas librería.

```
from typing import Protocol

class MemoryStore(Protocol):
    def recall(self, user_id: str, query: str) -> list[str]: ...
    def store(self, user_id: str, entry: dict) -> None: ...

class Brain:
    def __init__(self, memory: MemoryStore, safety: SafetyChecker):
        self.memory = memory
        self.safety = safety
```

Composition root en main.py — un solo lugar donde todo se conecta. Testing con fakes que satisfacen el Protocol sin herencia.

2. PROTOCOL vs ABC vs DUCK TYPING

Usar	Cuando
Protocol	Quieres type checking SIN forzar herencia. Default choice.
ABC	Necesitas forzar implementación Y tienes lógica compartida en la base.
Duck typing	Scripts rápidos, helpers internos. No para fronteras de módulo.

MARIS: Protocol en todas las fronteras de módulo (safety, memory, flow, core).

3. STRATEGY PATTERN → REEMPLAZAR IF/ELSE CHAINS

Directamente aplicable a resolve_state / select_mode:

```
class ModeStrategy(Protocol):
    def matches(self, state: UserState) -> bool: ...
    def execute(self, ctx: PipelineContext) -> PipelineContext: ...

class ModeResolver:
    def __init__(self, strategies: list[ModeStrategy]):
        self.strategies = strategies
    def resolve(self, state: UserState) -> ModeStrategy:
        for s in self.strategies:
            if s.matches(state):
                return s
        return CasualMode()
```

Composition root define el orden de prioridad:

```
resolver = ModeResolver([CrisisMode(), OnboardingMode(), SupportMode(),
CasualMode()])
```

Cada modo es independientemente testeable. Agregar uno nuevo = 1 clase + 1 línea.

4. PROJECT STRUCTURE (src layout)

```
src/maris/
├── main.py              # composition root
├── config.py            # Pydantic Settings
├── pipeline/
│   ├── context.py       # PipelineContext dataclass
│   ├── runner.py        # PipelineRunner
│   └── steps/           # cada paso = 1 archivo
├── safety/
│   ├── protocols.py     # SafetyChecker Protocol
│   └── evaluator.py
├── memory/
│   ├── protocols.py
│   └── supabase_store.py
├── brain/
│   ├── protocols.py
│   └── modes/
└── shared/
    ├── types.py
    └── errors.py
```

- `protocols.py` en cada módulo define qué espera del mundo exterior
- `main.py` es el único que conoce todas las implementaciones concretas
- Módulos nunca importan implementaciones de otros, solo protocols

5. CONFIGURACIÓN

Pydantic Settings > dataclasses > raw env vars:

```
class Settings(BaseSettings):
    supabase_url: str
    safety_threshold: float = Field(default=0.85)
    enable_physics_brain: bool = False
    model_config = {"env_file": ".env", "env_prefix": "MARIS_"}
```

Datos internos: frozen dataclasses:

```
@dataclass(frozen=True, slots=True)
class SafetyResult:
    is_safe: bool
```

```
    crisis_level: float
    requires_veto: bool = False
```

6. REFACTORING WORKFLOW

1. **Identify smell** — long method, feature envy, shotgun surgery, large if/else
2. **Extract** — cada responsabilidad a su propia clase con interfaz clara
3. **Inject** — dependencias por constructor via Protocol
4. **Test** — cada pieza en aislamiento con fakes

7. SOLID EN PYTHON

- **S**: Cada pipeline step hace UNA cosa
- **O**: ModeResolver abierto a extensión (nueva clase), cerrado a modificación
- **L**: Cualquier clase que satisfaga Protocol puede sustituir a otra
- **I**: No un Protocol gordo — separar: MemoryReader / MemoryWriter
- **D**: Módulos de alto nivel dependen de Protocols, no de implementaciones concretas

8. MODERN PYTHON

- Type hints siempre en fronteras públicas
- `dataclass` para datos, class para comportamiento
- `str | None` no `Optional[str]`
- `slots=True` en dataclasses
- `StrEnum` para choices fijos, no string literals
- `match` statements para pattern matching simple

PLAN CONCRETO PARA MARIS

1. Definir `PipelineContext` dataclass que fluye por los 23 pasos
2. Cada paso = clase implementando `PipelineStep` Protocol
3. Reemplazar resolve_state/select_mode if/else con Strategy + ordered resolver

4. Protocols en cada módulo. Runner solo conoce `PipelineStep`
5. Wiring en main.py (composition root)
6. Test cada step con fakes — sin Supabase, sin Claude, sin sentence-transformers

Primeagen + Code Aesthetics — Code Reading & Readability

Principios para revisar y refactorizar el backend de MARIS

PRIMEAGEN: NAVEGACIÓN DE CODEBASES

Método "Entry Point + Trace"

No leer top-to-bottom. Encontrar el entry point (route handler) y trazar UN path completo por el sistema. - Marcar cada módulo: I/O boundary, pure transform, o side effect - Solo después de trazar un path completo, leer branching logic - **Test:** Si no puedes explicar qué pasa con un mensaje desde HTTP request hasta response en <2 min, el código está enredado

Abstracción: "La abstracción incorrecta es peor que la duplicación"

- No abstraer hasta tener 3+ casos concretos (Rule of Three)
- Si tienes que leer la implementación de la abstracción para entender al caller, la abstracción falló
- **"Indirection is not abstraction"** — wrappear algo en función/clase sin simplificar el modelo mental es esconder complejidad
- Para MARIS: crisis_detector, veto_module, safety_filter pueden tener patrones similares pero lógica distinta. Mantenerlos separados ES correcto.

Testing: "Test Behavior, Not Implementation"

- Test boundaries (input/output del sistema), no wiring interno
- Integration tests > unit tests para sistemas reales
- Si refactorizar rompe tus tests pero no tu comportamiento, tus tests están mal
- No testear métodos privados — si son tan complejos, extráelos con interfaz pública
- Para MARIS: VERA tests mandan mensaje completo y assertan sobre response (tono, safety flags, crisis) — NO sobre si step 12 llamó a step 13

"Boring Code Wins"

- Código clever es liability — si toma >5s entender una línea, reescribir
- Explícito > implícito — 40 líneas obvias de if/elif > 10 líneas de metaclass dispatch
- State es el enemigo — cada estado mutable es un bug potencial
- "El mejor código es código que puedes borrar" — módulos que se pueden remover sin cascading failures

---

CODE AESTHETICS: LEGIBILIDAD

Never Nesting (Guard Clauses)

```
# MAL: 4 niveles de indentación
def process(msg, user):
    if user is not None:
        if user.is_active:
            if not user.is_banned:
                return do_work(msg)
    return None

# BIEN: guard clauses, happy path al nivel 0
def process(msg, user):
    if user is None: return None
    if not user.is_active: return None
    if user.is_banned: return None
    return do_work(msg)
```

Regla: Happy path al nivel de indentación más a la izquierda. Max 2 niveles en cualquier función.

Naming

- **Funciones = verbos:** `detect_crisis()`, `evaluate_safety()` — NO `crisis_handler()`
- **Booleans = preguntas:** `is_in_crisis`, `should_escalate` — NO `crisis_flag`
- **No nombres genéricos:** `data`, `result`, `info` no dicen nada. Usar `user_mood_vector`, `safety_verdict`
- **Nombrar el QUÉ, no el CÓMO:** `calculate_emotional_distance()` NO `run_cosine_similarity()`

Estructura de Funciones (regla del periódico)

- Signature = headline (QUÉ), primeras líneas = WHY, body = HOW
- Una función = UNA cosa en UN nivel de abstracción
- Max 20-25 líneas por función
- Max 3-4 parámetros — si más, usar dataclass
- Return type obvio por el nombre de la función

---

10 REGLAS PARA REVIEW DEL PIPELINE

1. Trazar un mensaje end-to-end antes de leer módulos aislados
2. Cada paso: guard clauses arriba, happy path sin nesting, max 2 niveles
3. Steps = stateless transforms: `step(context) -> context`
4. Test en boundaries: mensaje in, respuesta out. Mock solo APIs externas
5. Booleans como preguntas: `is_in_crisis` no `crisis_flag`
6. Funciones como verbos: `detect_`, `evaluate_`, `resolve_`, `select_`
7. Max 25 líneas/función, max 4 parámetros
8. No abstraer hasta tener 3 casos — duplicación en safety es OK si la lógica es distinta
9. Si puedes borrar un módulo sin tocar otros, la arquitectura está bien
10. Todos los steps siguen el mismo patrón estructural — consistencia aburrida > variación clever

TECNICO: SYSTEM DESIGN Y BACKEND

ByteByteGo (Alex Xu) — System Design para MARIS

Decisiones arquitectónicas concretas

---

1. SSE STREAMING

- SSE para streaming AI (server→client). HTTP POST para enviar mensajes. NO WebSocket.
- Heartbeat `:keepalive\n\n` cada 15s (carriers móviles matan conexiones idle a 30-60s)
- `Last-Event-ID` para reconnection — numerar eventos, buffer últimos N en Redis 5min TTL
- Backpressure: `asyncio.Queue(maxsize=100)` por conexión
- Multi-instancia futuro: Redis Pub/Sub para fan-out

2. CACHING

Pattern	Uso en MARIS
Cache-Aside	User profiles, anchors, crisis state lookups
Write-Through	Crisis state updates (cache + DB atómico)
Write-Behind	Analytics, session logs, message history

Claude API: Semantic cache con key = `intent + entities + user_state`. Redis 1hr TTL. Hit rate esperado 15-25%. **Embeddings:** Cache vector por input `sha256(text)`. Redis 24hr TTL. **Crisis:** NUNCA cachear — siempre computar fresh. **Invalidation:** TTL como safety net, invalidación explícita como mecanismo primario.

3. RATE LIMITING

- **Token Bucket** per-user: Free 10 msg/hr, Paid 60 msg/hr. Permite bursts (usuario en distress).
- **Sliding Window** global: Max 100 Claude API calls/min. Si excede → queue, no reject.

4. API PATTERNS

- Versioning URL: `/api/v1/chat`
- Response envelope: `{"status": "ok", "data": {...}, "meta": {"request_id": "xxx"}}`
- **Idempotency keys:** ULID-based. Redis 5min TTL. Previene mensajes duplicados por retry de iOS.
- Timeout cascade: iOS 30s → Gateway 25s → Claude 20s. Inner siempre < outer.

5. DATABASE (Supabase/Postgres)

- **Usar port 6543** (PgBouncer transaction mode), NO 5432 (directo)
- Pool max 15-20 conexiones para Railway
- Indexes necesarios: `sql CREATE INDEX idx_messages_user_created ON messages(user_id, created_at DESC); CREATE INDEX idx_crisis_state_user ON crisis_state(user_id) WHERE active = true;`
- **asyncio.gather** para queries paralelas (1 round trip vs 5)

- Nunca `SELECT *` — especificar columnas
- Cursor pagination (`WHERE id > last_id`), no OFFSET

6. COLD START (el problema de 6 segundos)

- Health check gating: `/health` retorna 503 hasta que modelo esté listo. Railway no rutea tráfico hasta 200.
- Warm model con dummy inference en startup
- Ping cada 5min para evitar sleep (cron desde Supabase Edge Functions)
- Lazy loading para modelos no-críticos

7. ASYNC PROCESSING

Pipeline split:

```
Steps 1-8 (síncronos): Intent → Safety → State → Mode
Steps 9-15 (parallelizable): Anchors ∥ History ∥ Health data
Steps 16-20 (secuenciales): Prompt → Claude → Stream
Steps 21-23 (fire-and-forget queue): Save DB → Update state → Analytics
```

Steps 21-23 al queue = response más rápido al usuario.

8. PRIORIDAD DE IMPLEMENTACIÓN

1. Health check gating (30min)
2. Redis cache user data (2hrs)
3. asyncio.gather queries paralelas (1hr)
4. Fire-and-forget queue steps 21-23 (1hr)
5. Token bucket rate limiting (2hrs)
6. SSE heartbeat + Last-Event-ID (2hrs)
7. Semantic cache Claude responses (4hrs)
8. Idempotency keys (1hr) Total: ~14 horas para ir de "funciona" a "production-grade"

Codely (DDD/Hexagonal) + Hussein Nasser (Backend Engineering)

Decisiones arquitectónicas para MARIS

CODELY: ARQUITECTURA

Hexagonal para MARIS: Parcial, no completa

Crear ports SOLO donde hay un segundo adapter realista: - **SÍ port:** `LLMProvider` (Claude hoy, Gemini mañana), `EmbeddingProvider` (sentence-transformers hoy, OpenAI mañana) - **NO port:** Supabase auth (no va a cambiar), NotificationService (solo APNs)

3 Bounded Contexts naturales de MARIS

Contexto	Responsabilidad	Entidades clave
Conversation	Chat flow, intent routing, response gen	Message, Session, Intent, Response
Clinical	Safety, crisis detection, mood tracking	CrisisState, MoodVector, RiskLevel
Memory	Anchors, user history, semantic search	Anchor, UserProfile, EmbeddingVector

"User" significa algo diferente en cada contexto = bounded contexts separados. **Mantenerlos como módulos en un monolito.** Bounded context = frontera lógica, NO de deployment.

Value Objects vs Entities

```
# Value Object: inmutable, comparado por valor, sin identidad
@dataclass(frozen=True)
class MoodScore:
    value: float
    def __post_init__(self):
        if not -1.0 <= self.value <= 1.0:
```

```
            raise ValueError(f"MoodScore debe estar entre -1 y 1")

# Entity: tiene identidad, mutable
@dataclass
class UserSession:
    id: str  # UUID = identidad
    user_id: str
    mood_score: MoodScore
    crisis_state: CrisisState
```

Value Objects MARIS: MoodScore, RiskLevel, EmbeddingVector, TokenCount Entities MARIS: UserSession, Anchor, ConversationThread, CrisisEvaluation

Testing Hexagonal

- Unit test: Application layer con ports mockeados
- Integration test: adapters contra servicios reales
- NO testear domain logic a través del API

HUSSEIN NASSER: BACKEND

SSE vs WebSocket vs Long Polling

Decisión: SSE. Chat de MARIS es asimétrico. User manda mensaje corto (POST). AI streama respuesta larga (SSE). No necesitas bidireccional. - SSE auto-reconnecta (WebSocket no) - SSE funciona por todos los proxies sin config - FastAPI tiene soporte nativo via StreamingResponse

Connection Pooling Supabase

- Cada conexión Postgres = fork de proceso OS (~10MB RAM)
- **Usar port 6543** (PgBouncer transaction mode)
- Pool max 15-20 para Railway
- Transaction mode, no session mode (endpoints son short-lived)

Postgres Optimization

1. Nunca `SELECT *`

2. Nunca `OFFSET` para paginación — cursor-based con `WHERE id > last_id`
3. Composite indexes para queries frecuentes
4. `EXPLAIN ANALYZE` en cada query que corre >1 vez por request
5. Evitar `SELECT COUNT(*)` — full table scan en Postgres

Idempotency

ULIDs como idempotency keys (ordenados + index-friendly). Redis 5min TTL. Crítico porque: Claude API cuesta dinero (duplicados = tokens desperdiciados), network retries en iOS, double-tap de usuarios, Railway puede reiniciar containers mid-request.

Diagnóstico de Performance (framework)

1. Communication: ¿bottleneck en protocolo?
2. Execution: ¿bottleneck en tu código?
3. Proxying: ¿bottleneck en intermediarios?
4. Caching: ¿re-computando cosas cacheables?
5. Database: ¿queries lentos?

Jerarquía de bottleneck en MARIS: 1. Claude API latency (~2-5s) — no puedes hacer nada excepto streamear 2. Embedding computation (~200ms) — cachear agresivamente 3. Supabase queries (~50-100ms) — indexar, connection pooling 4. FastAPI overhead (~5ms) — negligible

BRANDING, NAMING Y DISEÑO VISUAL

Branding, Naming & Visual Design Frameworks

Sources

- The Futur - How Brand Strategy Works
- The Futur - Brand Strategy Fundamentals
- The Futur - Identity Design: Branding
- Jonathan Bell - How to Create a Great Brand Name (summary)
- How Brands Are Built - Types of Brand Names
- Brand Naming Examples & Frameworks
- Visual Hierarchy: F and Z Patterns (99designs)
- Color Psychology & 60-30-10 Rule
- Typography & Brand Personality
- Don Norman's Three Levels of Design (IxDF)
- Gestalt Principles (IxDF)
- Product Naming Process 2026 (InkBot)
- Startup Naming Trends 2025 (Frozen Lemons)

- Gestalt in Visual Branding (Crowdspring)

A. BRAND IDENTITY FRAMEWORK (Chris Do / The Futur)

Core Principle

> "A brand is a person's gut feeling about a product, service, or company. Branding is not what YOU say it is -- it's what THEY say it is." -- Marty Neumeier, quoted by Chris Do

The Order Matters: Strategy Before Design

1. **Strategy** (invisible) -- positioning, audience, voice, differentiation
2. **Identity** (visible) -- logo, colors, typography, imagery
3. **Experience** (felt) -- every touchpoint from app to support to packaging

Jumping to logo before strategy = decorating a house with no foundation.

Chris Do's Brand Strategy Process

1. **Discovery / Diagnostic** -- Why does this brand need to exist? What's happening in the market? Who is the customer? Not a "yes person" meeting -- honest advisory.
2. **Brand Attributes & Positioning** -- Facilitate word extraction from stakeholders. Boil down to ONE positioning phrase that captures the essence.
3. **Visual Exploration (Stylescape)** -- Collection of found images, patterns, marks, textures that REFLECT the brand attributes. Client approves direction before a single logo is drawn.
4. **Design Development** -- Sketching and drawing concepts that capture key attributes in the most interesting forms.
5. **Application & Mockups** -- Logo applied to contextually relevant surfaces (app screens, business cards, packaging).

Timeline: ~2 weeks discovery, 2-4 weeks design, 2 weeks finalization.

The Brand Brief (what you define BEFORE design)

Component	Question
Mission	Why do we exist beyond making money?
Vision	Where are we going in 5 years?
Values	What do we refuse to compromise on?
Personality	If the brand were a person, how would they talk?
Voice	Casual or formal? Direct or poetic? What words do we NEVER use?
Audience	Who specifically, and what do they care about?
Positioning	What makes us the ONLY option for this audience?
Promise	What do we guarantee every time?

Trust Triangle (Chris Do)

Three components that build brand trust: - **Authenticity** -- being who you say you are - **Empathy** -- genuinely caring about the user - **Logic** -- making sense in your approach

Content Ratio (9-1-1)

For every 9 pieces of value you give, 1 personal story, you earn 1 ask. Build trust through consistent delivery before asking for anything.

B. NAMING FRAMEWORK

Jonathan Bell's 7 Categories of Brand Names

Category	Definition	Examples	Strength	Weakness
Eponymous	Named after a person (founder, inventor)	Disney, Tesla, Adidas, Burberry	Authority, legacy	Tied to one person

Category	Definition	Examples	Strength	Weakness
Descriptive	Says what it does	American Airlines, Home Depot, YouTube, PayPal	Instant clarity	Hard to trademark, generic feel
Acronym	Shortened descriptive	GE, BMW, IBM, KFC, HSBC	Clean, short	Meaningless without awareness
Real Word (Suggestive)	Existing word that hints at benefit	Uber, Slack, Amazon, Apple	Memorable, metaphorical	Hard to own SEO
Composite	Two words combined	Facebook, Ray-Ban, FedEx, Instagram	Descriptive + catchy	Can feel dated
Invented	Modified/created word	Kleenex, Pinterest, Spotify	Unique, trademarkable	Requires marketing investment
Abstract	No inherent meaning, pure phonetics	Rolex, Kodak, Xerox	Total blank slate	Requires massive brand building

The Naming Matrix (How Brands Are Built)

Two axes: - **Vertical: Approach** -- Descriptive --> Suggestive --> Abstract - **Horizontal: Construct** -- Real Word | Compound | Coined

This creates 9 zones:

```
                  Real Word      Compound        Coined
Abstract      |  Apple, Nike  |               | Kodak, Xerox  |
Suggestive    |  Amazon, Uber | BestBuy       | Febreze       |
Descriptive   |  Founders'    | HomeDepot     | Acronyms      |
```

Bell's Rules for Great Names

1. **Names don't describe -- they STAND for a big idea** that translates into emotional appeal

2. **Reflect core values**: Nike = winning, GoPro = heroism, Apple = simplicity
3. **Check availability thoroughly** -- trademark + negative meanings in other languages
4. **Don't obsess over exact-match domains** -- search and social matter more now
5. **Think strategically** -- name should support vision beyond current product

Naming Best Practices (2025-2026 research)

- **Length**: 5-12 letters optimal. Under 4 syllables.
- **Pronounceability**: If someone can't say it after hearing it once, it fails
- **Spellability**: If someone can't type it after hearing it once, it fails
- **Distinctiveness**: Must not sound like a competitor
- **Cross-language check**: Test in your top 5 markets minimum
- **Domain flexibility**: .com not required -- .app, .co, .io all viable now
- **App Store**: Search for conflicts in iOS App Store and Google Play specifically

Context Rotation Technique

Take a word from one domain and apply it to another: - **Apple** (fruit --> tech) = simplicity, naturalness - **Amazon** (river --> retail) = vast, everything - **Python** (snake --> programming) = power, flexibility - **Slack** (leisure --> work tool) = ease, no pressure - **Uber** (German "above" --> transport) = superior This creates instant intrigue because the word has existing emotional weight but unexpected context.

2025-2026 Naming Trends

1. **Portmanteau names** -- blending two words (Pinterest = pin + interest, Instagram = instant + telegram)
2. **Single-word monikers** -- clean, one word (Stripe, Notion, Linear)
3. **Lowercase by default** -- brands launch in lowercase to feel approachable
4. **Nature/elemental words** -- calm, serene, flow (wellness space especially)
5. **Verb-like names** -- names that feel like actions (Zoom, Dash, Spark)

C. VISUAL HIERARCHY FOR CONVERSION

The Two Scanning Patterns

Z-Pattern (for minimal layouts, landing pages, app screens)

```
1. Top-left (logo/brand) -----> 2. Top-right (CTA/nav)
         \
          \   (diagonal scan)
           \
3. Bottom-left (supporting info) -> 4. Bottom-right (final CTA)
```

Best for: landing pages, app home screens, onboarding screens, ads. Rule: Place your PRIMARY action at position 2 or 4.

F-Pattern (for content-heavy pages)

```
1. Top horizontal scan ========================
2. Second horizontal scan ============
3. Vertical scan down left side
   |
   |
   |
```

Best for: blog posts, news, settings screens, lists. Rule: Front-load the first two words of every line/heading.

Visual Hierarchy Tools (ranked by power)

1. **Size** -- bigger = more important. Period.
2. **Color/Contrast** -- bright on muted background wins attention
3. **Position** -- top-left gets seen first (Western reading)
4. **Whitespace** -- isolation = importance. Give key elements breathing room.
5. **Typography weight** -- bold vs. regular creates instant hierarchy
6. **Imagery** -- humans look where faces look (eye-tracking proven)
7. **Motion** -- animation draws the eye but use sparingly

Color Psychology

Color	Association	Use When
Blue	Trust, calm, stability, professionalism	Finance, health, tech, corporate
Green	Growth, health, nature, money	Wellness, sustainability, finance
Purple	Creativity, luxury, wisdom	Premium products, creative tools
Red	Energy, urgency, passion, danger	CTAs, alerts, food, entertainment
Orange	Warmth, enthusiasm, friendliness	Casual brands, calls-to-action
Yellow	Optimism, clarity, warmth	Accents, warnings, cheerful brands
Black	Sophistication, luxury, power	Luxury, fashion, minimalism
White	Purity, simplicity, space	Health, tech, minimalist brands
Teal/Cyan	Balance, clarity, refreshment	Wellness, modern tech

Color increases brand recognition by up to 80% (research-backed).

The 60-30-10 Color Rule

- **60%** -- Dominant color. Background, large surfaces. Sets the mood.
- **30%** -- Secondary color. Supporting elements, cards, sections. Creates contrast.
- **10%** -- Accent color. CTAs, highlights, badges. Draws the eye.

Example: McDonald's = 60% red, 30% yellow, 10% white.

Typography Psychology

Type	Communicates	Examples	Best For
Serif	Trust, tradition, authority	Times, Garamond, Merriweather	Established brands, editorial
Sans-serif	Modern, clean, approachable	Helvetica, Inter, SF Pro	Tech, apps, startups

Type	Communicates	Examples	Best For
Rounded sans	Warmth, friendliness, safety	Nunito, Varela Round, Quicksand	Wellness, children, casual
Geometric sans	Precision, contemporary	Futura, Montserrat, Poppins	Design-forward, modern
Monospace	Technical, code, precision	JetBrains Mono, Fira Code	Dev tools, data
Handwritten/Script	Personal, creative, human	Caveat, Dancing Script	Invitations, personal brands

Key stat: Serif fonts increase perceived trustworthiness by 40%. Rounded fonts increase perceived warmth.

Weight hierarchy in practice: - **Bold/Black** -- headlines, key numbers, primary actions - **Semibold/Medium** -- subheadings, labels, secondary emphasis - **Regular** -- body text, descriptions - **Light** -- captions, metadata, tertiary info

---

D. DESIGN PRINCIPLES THAT SELL

Simplicity (Less = More Conversion)

- Every additional element on screen reduces conversion by forcing choice
- Hick's Law: time to decide increases logarithmically with number of options
- Rule: One screen, one purpose, one primary action
- Apple, Google, Stripe all maximize whitespace

Alignment and Grid Systems

- 8pt grid: all spacing and sizing in multiples of 8 (8, 16, 24, 32, 40, 48...)
- Consistent alignment creates unconscious trust
- Misalignment, even by 1-2px, creates subconscious unease
- iOS uses the 8pt grid natively in SF design system

Gestalt Principles Applied to Product Design

Principle	Rule	Application
Proximity	Things close together are perceived as related	Group related controls. Separate unrelated sections with whitespace.
Similarity	Things that look alike are perceived as related	Use consistent card styles for same-type content. Different styles = different function.
Closure	Brain completes incomplete shapes	Progress rings, partial circles, logo design (Apple logo, WWF panda)
Continuity	Eye follows smooth paths	Horizontal scroll, timeline views, onboarding flows
Figure/Ground	Brain separates foreground from background	Modal sheets, overlays, blurred backgrounds behind content
Common fate	Things moving together are perceived as grouped	Animations where related elements move in sync

Don Norman's 3 Levels of Emotional Design

Level 1: VISCERAL (first impression, automatic) - What it looks like before you use it - Color, shape, texture, feel - "Do I want to pick this up / tap this?" - Design action: Beautiful screenshots, polished icon, smooth animations - For MARIS: The app icon, the first screen, the color palette

Level 2: BEHAVIORAL (during use, subconscious) - Does it work? Is it easy? Does it feel good to use? - Responsiveness, flow, predictability - "Did that do what I expected?" - Design action: Fast transitions, clear feedback, intuitive navigation - For MARIS: Chat feels instant, dashboard loads fast, plans are easy to create

Level 3: REFLECTIVE (after use, conscious) - What does using this say about me? - Status, identity, meaning, memory - "I'm the kind of person who uses this" - Design action: Brand story, social proof, personal growth narrative - For MARIS: "I use this to protect my focus" -- identity statement, not utility

The reflective level is where brand loyalty lives. Branding is a reflective-level exercise.

E. APPLIED ANALYSIS: MARIS NAME EVALUATION

Current Situation

- Name: **MARIS**
- Origin: No specific meaning; chosen intuitively
- Letters: 5 (optimal range 5-12)
- Syllables: 2 (under the 4-syllable max)
- Pronounceable: Yes (MAH-ris)
- Spellable: Yes
- Cross-language: "Maris" = "of the sea" in Latin. No negative connotations found.
- New positioning: "Focus shield" -- protects your mental space

Evaluation by Jonathan Bell's Framework

MARIS falls into **Category 4: Real Word (Suggestive)** -- it's a real Latin word that doesn't describe the product but has evocative qualities.

Criterion	Score	Notes
Short and memorable	9/10	5 letters, 2 syllables -- excellent
Easy to pronounce	8/10	Clear in English/Spanish. Slight ambiguity (MARE-iss vs MAH-ris)
Easy to spell	9/10	No trick letters, no ambiguity
Emotional resonance	7/10	Sea/water connotations = calm, depth. Fits wellness.
Distinctiveness	8/10	Not a common word. Not a competitor name.
Trademarkable	7/10	Real Latin word -- harder than invented, but niche-specific is fine
Domain availability	?/10	Needs checking: maris.app, getmaris.com, trymaris.com
App Store clarity	6/10	"MARIS" alone doesn't tell you what it does. Needs subtitle.

Overall: 7.7/10 -- MARIS is a strong name for this positioning.

Why MARIS Works for "Focus Shield"

1. **Latin "of the sea"** -- Sea is the ultimate metaphor for calm and depth. People associate ocean with breathing, decompression, mental space.
2. **Context Rotation** -- Taking a nature/sea word into the productivity/focus space creates intrigue (same technique as Apple, Amazon, Slack).
3. **Not clinical** -- "MARIS" does NOT sound like a therapy app. It sounds like a tool, a companion, a space. This is exactly the repositioning goal.
4. **Gender-neutral** -- Works across demographics.
5. **International** -- Works in English, Spanish, Portuguese, French, Italian. Latin root is universal.

What MARIS Needs to Work as a Name

- **Subtitle/tagline** that explains immediately: "MARIS -- your focus shield" or "MARIS -- protect your mental space"
- **Visual identity** that reinforces calm-focus, not clinical-therapy
- **App Store subtitle**: "Focus & Mental Clarity" or "Protect Your Day"
- **Never** position as "AI therapy" or "mental health chatbot" in marketing

Alternative Names Evaluated (for comparison)

Name	Type	Pros	Cons	vs MARIS
Cove	Real word/suggestive	Calm, protected, short	Common word, hard to own	Weaker -- too generic
Solis	Latin (sun)	Bright, clarity, energy	Sounds like solar company	Lateral -- different vibe
Haven	Real word/suggestive	Safety, protection	Overused in wellness	Weaker -- cliche
Nura	Invented	Unique, soft, short	No meaning, requires building	Neutral -- blank slate
Tidal	Real word/suggestive	Rhythm, natural, movement	Sounds like music (Tidal exists)	Weaker -- conflict

Name	Type	Pros	Cons	vs MARIS
Anchora	Latin (anchor)	Stability, grounding	Too literal, clinical hint	Weaker -- too on-the-nose

Recommendation: Keep MARIS. It scores well across all frameworks and already has brand equity in development.

F. VISUAL IDENTITY DIRECTION FOR MARIS

Brand Brief (filled in)

Component	MARIS Definition
Mission	Protect people's mental space so they can focus on what matters
Vision	The default tool people open to check in with themselves
Values	Privacy-first. Never prescriptive. Concrete over abstract.
Personality	Calm friend who's smarter than they let on. Not a therapist. Not a coach.
Voice	Direct, warm, short sentences. "Eso pesa. Cuanto hace?" Not "I sense some tension."
Audience	22-35, overwhelmed professionals/students who won't download a "therapy app"
Positioning	The only focus tool that actually understands how you feel today
Promise	We read the room so you don't have to explain yourself

Color System (60-30-10)

Direction: "Deep ocean calm" -- not pastel wellness, not clinical blue

Role	Color	Hex (suggested)	Rationale
60% Dominant	Deep navy / off-black	`#0D1B2A` or `#1B2838`	Authority without coldness. "Night ocean." Grounds everything.

Role	Color	Hex (suggested)	Rationale
30% Secondary	Warm slate / muted teal	`#415A77` or `#3A6B7E`	Depth, intelligence, calm. Not baby-blue wellness.
10% Accent	Soft gold / warm amber	`#D4A574` or `#E8B86D`	Warmth, insight, "moment of clarity." The star that appears for takeaways.

Why NOT pastel blue/green: Every wellness app uses them. MARIS needs to feel like a TOOL, not a meditation cushion. Dark backgrounds = focus mode (like code editors, dark mode, cinema). The target audience (22-35 overwhelmed professionals) already prefers dark mode.

Light mode alternative: Swap dominant to warm off-white `#F5F0EB` (paper, not screen-white), keep slate and gold. Same hierarchy.

Typography System

Role	Font	Weight	Why
Display / Headlines	SF Pro Display or Inter	Semibold	Clean, geometric, modern. iOS-native feel.
Body / Chat	SF Pro Text or Inter	Regular	Maximum readability at small sizes
Accent / Labels	SF Pro Rounded	Medium	Warmth without losing authority. Use for moods, tags, pills.
Numbers / Data	SF Mono or JetBrains Mono	Regular	Physics engine data, stats -- technical precision feel

Why rounded for accents only: Full rounded font = "children's app." Rounded ONLY on soft elements (mood dots, health pills, tags) creates contrast: the app is serious, but it sees you as human.

Imagery & Iconography Style

- **Icons**: Outlined, 1.5px stroke, rounded caps. NOT filled. Airiness > solidity.
- **Illustrations**: None. Zero. Let the data and whitespace speak.
- **Imagery**: Abstract gradients only -- deep ocean tones, never photos of people meditating.

- **Motion**: Slow, deliberate. Ease-in-out curves. Nothing bouncy. Think: breathing rhythm.
- **Haptics**: Light impact for confirmations. Medium for completions. Never aggressive.

Design System: "Calm Focus" Not "Clinical Therapy"

What to AVOID: - Pastel gradients (Calm, Headspace territory) - Heart icons, brain icons, smiley faces (therapy app signals) - The word "wellness" anywhere - Purple + pink combinations (too spiritual) - Bubble/cloud shapes (too soft, childish) - Stock photos of people journaling on mountains

What to EMBRACE: - Dark spaces with purposeful light (cinema, focus mode) - Cards with generous padding (breathing room = Gestalt proximity principle) - Single-line labels (not paragraphs) - The gold accent sparingly -- when it appears, it MEANS something (takeaway, insight, completion) - Horizontal rules and subtle separators (structure = trust) - Status through shape, not emoji (stability ring, capacity bar)

Norman's 3 Levels Applied to MARIS

Visceral (first 3 seconds): - App icon: Simple "M" lettermark in gold on dark navy. Or: abstract wave/shield shape. - Splash: Dark background, "MARIS" in light type, subtle gold underscore animation - First impression: "This looks like a premium tool, not a free therapy chatbot"

Behavioral (daily use): - Dashboard loads in <1s with today's data - Chat responds with streaming text (no loading spinner sitting) - Plans can be created in 1 tap (voice) or 3 taps (manual) - Night review is 2 taps for good days, 4 taps for heavy days - Physics engine data is visual, never numerical soup

Reflective (identity): - "I use MARIS to protect my focus" -- says something about the user - Not "I use a therapy app" -- stigma-free positioning - Weekly digest makes users feel SEEN, not analyzed - Sharing a takeaway feels like sharing wisdom, not vulnerability

App Store Presentation

Title: MARIS -- Focus Shield **Subtitle**: Protect your day. Understand your patterns. **Category**: Productivity (NOT Health & Fitness, NOT Medical) **Screenshots** (in order, Z-pattern logic): 1. Dashboard with today's data -- "Your day at a glance" 2. Chat in action -- "Check in. No

judgment." 3. Physics engine visualization -- "See your patterns" 4. Night review -- "Close the day in 30 seconds" 5. Plans completed with celebration -- "Get things done"

What the listing communicates: This is a focus and productivity tool that happens to understand emotions. Not a therapy app that happens to have a to-do list.

G. QUICK REFERENCE: FRAMEWORKS CHEAT SHEET

Branding Order of Operations

1. Strategy (who, why, what position)
2. Name (does it fit the strategy?)
3. Visual identity (does it FEEL like the strategy?)
4. Design system (can I apply it consistently?)
5. Content/marketing (does every touchpoint match?)

Name Evaluation Checklist

- [ ] Under 4 syllables
- [ ] 5-12 letters
- [ ] Pronounceable after hearing once
- [ ] Spellable after hearing once
- [ ] No competitor conflicts
- [ ] No negative meanings in target markets
- [ ] Domain or variant available
- [ ] App Store name available
- [ ] Supports a bigger idea (not just a feature)

Visual Hierarchy Checklist

- [ ] One primary action per screen
- [ ] Z-pattern for minimal layouts
- [ ] F-pattern for content-heavy screens
- [ ] 60-30-10 color distribution
- [ ] 8pt grid spacing

- [ ] Bold for primary, medium for secondary, regular for body
- [ ] Whitespace used to create grouping (Gestalt proximity)
- [ ] Accent color reserved for meaningful moments

Emotional Design Checklist

- [ ] Visceral: Does it look premium in the first 3 seconds?
- [ ] Behavioral: Does every action feel instant and predictable?
- [ ] Reflective: Does using this make the user feel good about themselves?

CRO Y A/B TESTING

CRO & A/B Testing — Frameworks Aplicados a MARIS

Fuentes: CXL, WiderFunnel (LIFT), Hotjar, Product-Led Growth

---

1. LIFT MODEL (WiderFunnel — Chris Goward)

6 factores de conversión. 1 vehículo + 2 drivers + 2 inhibidores:

VEHÍCULO: - **Value Proposition** — "¿Por qué elegir esto sobre alternativas, incluyendo no hacer nada?"

DRIVERS (aumentan conversión): - **Relevance** — ¿La página coincide con lo que el usuario esperaba? Mismatch = bounce - **Clarity** — ¿Entiende la oferta y la acción en 5 segundos? (diseño + copy) - **Urgency** — ¿Razón para actuar AHORA? Interna (necesidad propia) o externa (escasez, deadline)

INHIBIDORES (disminuyen conversión): - **Anxiety** — Miedos: privacidad, "¿es legítimo?", "¿funcionará?" - **Distraction** — Elementos que compiten por atención y alejan del CTA

LIFT aplicado a cada pantalla de MARIS:

Pantalla	Anxiety principal	Distraction principal
App Store	Privacidad con AI, reviews bajas	"You might also like" de la tienda
Onboarding	"¿Esto realmente funciona?"	Demasiados permisos, onboarding largo
Primer mensaje	"¿El AI es bueno?" — primera impresión	UI cluttered, no claro qué puede hacer
Paywall	"¿Puedo cancelar?"	Demasiadas opciones de plan

---

2. ResearchXL (CXL — Peep Laja)

Nunca testear ideas random. Investigar primero.

6 pilares en orden: 1. **Technical Analysis** — Crash logs, API latency, onboarding completion por device/iOS version 2. **Digital Analytics** — Funnel de onboarding por cohort, retention curves, feature usage 3. **Heuristic Analysis** — Walkthrough con LIFT, 3-5 evaluadores independientes 4. **Mouse/Interaction Tracking** — Session recordings (UXCam/Smartlook para iOS) 5. **Qualitative Surveys** — "¿Qué casi te detuvo?" "¿Qué falta?" "¿Qué te trajo aquí?" 6. **User Testing** — 5 usuarios task-based. Graba pantalla + audio. 5 usuarios capturan 85% de problemas.

Output: Findings → Hypotheses → Prioritize (PIE/ICE) → A/B Test

---

3. PRIORIZACIÓN: PIE vs ICE

PIE (para elegir QUÉ PÁGINA optimizar): - Potential + Importance + Ease / 3

ICE (para elegir QUÉ IDEA testear): - Impact × Confidence × Ease (multiplicativo)

PIE para MARIS:

Stage	P	I	E	PIE
App Store→Download	7	8	6	7.0
Download→First Message	**9**	**9**	**7**	**8.3**
First Message→Day 3	8	8	5	7.0
Day 3→Day 30	6	7	4	5.7
Free→Paid	**8**	**10**	**7**	**8.3**

Prioridad: Onboarding y Paywall empatados. Empezar con onboarding porque es upstream.

---

4. A/B TESTING ESTADÍSTICAS

Bayesian vs Frequentist

- **Frequentist:** Calcular sample size ANTES, correr hasta completar, NUNCA peek midway. Mejor para alto tráfico.
- **Bayesian:** Outputs probabilidad ("94% chance B es mejor"). Más práctico para bajo tráfico. Puede checar en cualquier momento (pero más data = más preciso).

Para MARIS early stage: Bayesian es más práctico.

Reglas de sample size:

- Mínimo ~1,000 conversiones totales entre todas las variantes
- Nunca correr test <7 días aunque tengas el sample size
- Mínimo 2 ciclos de negocio completos (2 semanas)
- Para suscripción: medir downstream (Day 30 retention) = tests de 30+ días

Errores comunes:

1. **Peeking** — Checar diario y parar cuando $p < 0.05$. Infla false positives de 5% a 30%+
2. **Parar temprano** — Esperar sample size Y duración mínima
3. **Muchas variantes** — Empezar con A/B (2), no A/B/C/D

4. **Testear cosas tiny** — Color de botón cuando tu value prop no es clara
5. **Sin hipótesis** — Ideas random sin research = waste de tráfico
6. **Segment mining post-hoc** — "No ganó overall pero ganó para iOS 17 los martes" = p-hacking

---

5. PROCESO CUALITATIVO HOTJAR

Heatmaps — qué buscar:

- **Rage clicks** — Tapping repetido en elementos no-interactivos
- **Dead zones** — Áreas con cero interacción donde hay CTAs
- **Scroll drop-off** — ¿Qué % llega debajo del fold? Info crítica debe estar arriba

Session recordings — señales de fricción:

- **U-turns** — Va a pantalla, regresa inmediatamente
- **Scrolling excesivo** — Busca algo que no encuentra
- **Pausas largas** — 10+ segundos sin interacción = confusión
- **Error states** — ¿Se recuperan o abandonan?

Surveys clave:

1. Al llegar: "¿Qué te trajo a MARIS hoy?"
2. En fricción: "¿Algo no funciona como esperabas?"
3. Post-primera interacción: "¿Qué casi te detuvo de probar MARIS?"
4. Al salir: "¿Qué te haría volver?"

Pipeline: Observe → Ask → Hypothesize → Prioritize (ICE) → Test

---

6. PRODUCT-LED ONBOARDING

Time-to-Value (TTV)

Tiempo desde signup hasta el primer "aha moment." **Para MARIS: Valor en el PRIMER MENSAJE. No después de 3 días. No después de 5 pantallas de onboarding.**

Si el onboarding tarda 5 pantallas antes de que el usuario pueda escribir, TTV es demasiado largo. Considerar: dejar que escriban inmediato y recoger data contextualmente.

Activation Metric

La acción que más correlaciona con retención a Day 30.

Candidatos MARIS: - ¿Mandó primer mensaje? (muy básico) - ¿Completó sesión de 15+ min de enfoque? - ¿Regresó Day 2? - ¿Usó MARIS 3 veces en primeros 7 días? - ¿Activó notificaciones?

Necesitas datos para encontrar TU "magic number." (Facebook: 7 amigos en 10 días. Slack: 2,000 mensajes. MARIS: ?)

BJ Fogg: Behavior = Motivation × Ability × Trigger

Si usuarios no convierten en un paso: motivación baja, es muy difícil, o falta el trigger.

ROADMAP DE TESTS — PRIORIZADO POR ICE

#	Test	ICE	Stage
1	Reducir onboarding a 2 pantallas + primer mensaje inmediato	504	Onboarding
2	Reescribir primera respuesta de AI para valor instantáneo	504	Activación
3	Notificación Day 1 personalizada vs genérica	448	Activación
4	Timing del paywall: post-momento-de-valor vs time-based	392	Monetización
5	Screenshots App Store: outcome vs feature	384	ASO
6	Diferir permisos hasta después de entregar valor	336	Onboarding
7	Price point: $4.99 vs $9.99 vs packs	280	Monetización
8	Reporte semanal de progreso	210	Retención

#	Test	ICE	Stage
9	Optimización horario check-in mañanero	240	Retención
10	Streak/gamification vs sin gamification	120	Retención

ANALYTICS MÍNIMO VIABLE

Eventos a instrumentar antes de cualquier test:

```
# Onboarding
app_opened (first_time: bool)
onboarding_screen_viewed (screen_number: int)
onboarding_completed
permission_granted/denied (type)

# Activación
first_message_sent
first_response_received
session_completed (duration: seconds)
returned_day_N (N: 1,2,3,7,14,30)

# Engagement
message_sent (session_number, day)
notification_tapped/dismissed

# Monetización
paywall_viewed (trigger: string)
purchase_completed (product_id, price)
trial_started/converted/expired
subscription_cancelled
```

PRINCIPIO UNIFICADOR

Acortar time-to-value. Que el usuario llegue a su primera interacción significativa con el AI lo más rápido posible. Esto afecta cada métrica downstream.

FINANZAS PARA FUNDADORES

Finanzas para Fundadores — MARIS Financial Model

Fuentes: YC, Profit First, Lean Startup, OKRs, Damodaran

---

1. UNIT ECONOMICS DE MARIS

```
============================================
MARIS UNIT ECONOMICS ($9.99/mo)
============================================
Revenue/user:       $9.99/mo
Apple Cut (15%):    -$1.50
API Cost (Haiku):   -$0.25
Infra (amortized):  -$0.20
--------------------------------------------
GROSS PROFIT/USER:  $8.04/mo (80.5% margin)
--------------------------------------------
Churn rate:         7% mensual
Avg Lifespan:       14.3 meses (1/0.07)
LTV:                $142.86
GM-adjusted LTV:    $115.00
Target CAC:         < $38 (para ratio 3:1)
```

```
Payback @ $10 CAC:  1.2 meses
==========================================
```

Fórmulas clave

- **LTV** = ARPU × (1 / Churn Rate) = $9.99 × 14.3 = $142.86
- **CAC** = Gasto total adquisición / Nuevos clientes
- **LTV:CAC** > 3:1 = negocio saludable. MARIS a $10 CAC = 11.5:1
- **MRR** = Usuarios pagados × ARPU
- **ARR** = MRR × 12
- **NRR** (Net Revenue Retention) — >100% = creces sin nuevos usuarios (necesita upsell tiers)
- **Payback period** = CAC / (ARPU × Gross Margin)

Path a $1M ARR

- $1M ARR = $83,333 MRR = **8,342 suscriptores**
- A 20% MoM growth desde 50 usuarios: ~21 meses
- A 15% MoM: ~24 meses
- A 10% MoM: no llegas (default dead)

---

2. BURN RATE & RUNWAY

Costos fijos mensuales (solo dev, pre-revenue)

Gasto	Costo/mes
Apple Developer	$8.25
Railway	$20
Supabase	$0 (free tier)
Claude API (dev)	~$10
Dominio/misc	~$5

Gasto	Costo/mes
Total infra	**$43/mes**

Default Alive or Default Dead (Paul Graham)

Con $15K cash, 50 usuarios iniciales, 20% MoM growth: - **Break-even: ~mes 9-10** (~205 usuarios pagados, MRR > $2,043) - Cash nunca llega a cero → **DEFAULT ALIVE**

Sensibilidad: - 20% MoM → break-even mes 10, sobrevives - 15% MoM → break-even mes 12, apenas - 10% MoM → break-even mes 17, **DEFAULT DEAD** (sin cash en mes 13)

La palanca que importa: growth rate. No recortar costos ($43/mes ya es mínimo).

---

3. PROFIT FIRST (Michalowicz)

Fórmula invertida: Sales - Profit = Expenses (no Sales - Expenses = Profit)

Allocations a $10K MRR ($8,230 Real Revenue)

Cuenta	%	Mensual
Profit	5%	$411
Owner's Pay	50%	$4,115
Tax	15%	$1,235
OpEx	30%	$2,469

5 cuentas bancarias

1. **INCOME** — Todo llega aquí
2. **PROFIT** — 5% intocable. Distribución trimestral como premio
3. **OWNER'S PAY** — 50%. Tu salario. De esto vives
4. **TAX** — 15%. Para impuestos estimados trimestrales

5. **OPEX** — 30%. Railway, API, marketing, herramientas, todo lo demás

Fase 1 (pre-$5K MRR): Profit 1%, Owner 60%, Tax 15%, OpEx 24% **Fase 2 ($5K-$20K):** Profit 5%, Owner 50%, Tax 15%, OpEx 30% **Fase 3 ($20K+):** Profit 10%, Owner 35%, Tax 15%, OpEx 40%

---

4. INNOVATION ACCOUNTING (Lean Startup)

Vanity Metrics vs Actionable Metrics

VANITY (evitar)	ACTIONABLE (trackear)
Total downloads	Weekly active users
Total registrados	Activated users (onboarding + 3 sesiones)
Total mensajes	Mensajes por usuario activo por semana
Rating App Store	NPS de survey in-app
Followers	Trial-to-paid conversion rate
Revenue acumulado	MRR y MoM growth rate

3 Engines of Growth — MARIS

1. **STICKY (primario):** Retención > churn. MARIS construye attachment emocional. Target: churn <5%
2. **VIRAL (secundario):** Word of mouth, referrals. Limitado por privacidad del tema. Referral program: ambos reciben mes gratis
3. **PAID (terciario, fase escala):** Solo cuando unit economics estén probados. App Store Search Ads

Build-Measure-Learn financiero

- Build: Launch $9.99/mo con 7-day free trial
- Measure: Trial-to-paid conversion (target >10%)

- Learn: Si es 3%, precio alto O trial corto O onboarding no demuestra valor
- Next build: Test $4.99/mo O 14-day trial O sesión guiada
- NUNCA cambiar múltiples variables a la vez

---

5. OKRs Q2 2026 (Abril-Junio)

Objetivo 1: Validar Product-Market Fit

Key Result	Target	0.7 =
100 suscriptores pagados para 30 junio	100	70
D30 retention >35% cohorte abril	35%	24.5%
Trial-to-paid conversion >8%	8%	5.6%
NPS >40 (n>30)	40	28

Objetivo 2: Unit Economics Sostenibles

Key Result	Target	0.7 =
Gross margin/user >75%	75%	52.5%
API cost/user <$0.50/mes	$0.50	$0.71
Alcanzar $1,000 MRR	$1,000	$700

Objetivo 3: Motor de Crecimiento

Key Result	Target	0.7 =
Identificar top 2 canales de adquisición por CAC	2	1
Publicar 8 piezas de contenido	8	5-6
500 impresiones/semana en App Store orgánico	500	350

Scoring: 0.7 = target ideal. Si llegas a 1.0 siempre, tus metas son muy fáciles.

---

6. VALUACIÓN EARLY-STAGE

Por qué DCF no funciona para MARIS ahora

Damodaran: DCF requiere proyectar flujos de caja, tasas de crecimiento y descuento. Para startups pre-revenue, los 3 inputs son guesses. La varianza es tan alta que el output no significa nada.

Métodos aplicables

Método	En $120K ARR	En $1M ARR
Revenue Multiple (2-3x early, 5x growing)	$240K	$5M
Comparable transactions (wellness apps)	$240K-$480K	$3-5M
SDE Multiple (bootstrapped exit, 3-5x)	N/A	$1.5-2.5M

Lo que importa para bootstrapped

No necesitas valuation para fundraising (tu burn es $43/mes). Lo que importa: - **SDE** (Seller's Discretionary Earnings) para un exit eventual - A $1M ARR con 50% SDE margin: SDE = $500K → venta a $1.5-2.5M

GROWTH MATH: TABLA DE MESES A $1M ARR

Mes	MRR	Usuarios	Growth
0	$0	0	—
3	$500	50	—
6	$2,500	250	30%

Mes	MRR	Usuarios	Growth
9	$8,000	800	25%
12	$20,000	2,002	20%
15	$40,000	4,004	15%
18	$65,000	6,507	12%
21	$83,333	8,342	10%

La palanca #1 es growth rate. Todo lo demás es secundario cuando tu burn es $43/mes.

CONCURRENCIA Y ASYNC

Python Concurrency, Asyncio & Async Architecture Patterns

Aplicado al backend de MARIS (FastAPI en Railway, SSE streaming, 5 background tasks, PyTorch)

Fuentes: ArjanCodes, Hussein Nasser, Cosmic Python (Percival & Gregory), Real Python, SuperFastPython, FastAPI docs, Python 3.11+ docs

---

1. CUÁNDO USAR QUÉ: THREADING vs MULTIPROCESSING vs ASYNCIO

Tipo de trabajo	Herramienta	Por qué	Ejemplo MARIS
I/O-bound, muchas conexiones	`asyncio`	Un solo thread, sin overhead de context switch, escala a miles	Supabase queries, Claude API calls, SSE streaming

Tipo de trabajo	Herramienta	Por qué	Ejemplo MARIS
I/O-bound, librería sync	`run_in_executor` con ThreadPool	No bloquea el event loop, GIL se libera durante I/O	Anthropic sync SDK (claude_sse_stream)
CPU-bound, GIL se libera	`threading` / `run_in_executor`	PyTorch/NumPy liberan el GIL durante operaciones de tensor	sentence-transformers embedding
CPU-bound puro Python	`multiprocessing`	Único way de usar múltiples cores en Python	NO aplica a MARIS (no tenemos CPU-bound puro)

Regla de oro:

```
¿Espera I/O?           → asyncio (async/await)
¿Librería sync que espera I/O?  → run_in_executor(ThreadPool)
¿CPU en C extensions (PyTorch)?  → run_in_executor(ThreadPool) — GIL released
¿CPU puro Python (loops)?       → ProcessPoolExecutor
```

Por qué PyTorch funciona con threading:

PyTorch libera el GIL durante operaciones de tensor (matmul, embeddings, forward pass). Esto significa que `sentence-transformers` embedding NO bloquea otros threads mientras computa. Un ThreadPoolExecutor es suficiente — NO necesitas multiprocessing.

---

2. ASYNCIO PATTERNS PARA FASTAPI

2.1 async/await basics — qué bloquea y qué no

```
# CORRECTO — no bloquea el event loop
async def get_user_data(user_id: str):
    # httpx es async-native, cada await cede control al event loop
    async with httpx.AsyncClient() as client:
        response = await client.get(f"/users/{user_id}")
    return response.json()
```

```
# INCORRECTO — bloquea TODO el event loop
async def get_user_data_bad(user_id: str):
    # requests es sync, bloquea el thread del event loop
    response = requests.get(f"/users/{user_id}")  # MUERTE
    return response.json()

# CORRECTO — sync code via executor
async def get_user_data_sync_lib(user_id: str):
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(
        None,  # usa default ThreadPoolExecutor
        lambda: requests.get(f"/users/{user_id}").json()
    )
    return result
```

Regla: Cada vez que llamas algo sin `await` dentro de una función `async`, estás bloqueando el event loop. Todas las demás requests se congelan.

2.2 asyncio.gather() — I/O paralelo

Ejecuta múltiples coroutines concurrentemente. Ideal cuando necesitas N resultados antes de continuar.

```
# MARIS: cargar datos del usuario en paralelo al inicio del pipeline
async def load_user_context(user_id: str) -> PipelineContext:
    # Estas 4 queries corren AL MISMO TIEMPO
    profile, anchors, history, crisis = await asyncio.gather(
        supabase.get_profile(user_id),
        supabase.get_anchors(user_id),
        supabase.get_recent_messages(user_id, limit=10),
        supabase.get_crisis_state(user_id),
    )
    return PipelineContext(
        profile=profile,
        anchors=anchors,
        history=history,
        crisis_state=crisis,
    )
```

Problema de gather: Si una task falla, las demás siguen corriendo en background (zombies).

```
# gather con return_exceptions=True — no cancela las demás
results = await asyncio.gather(
```

```
    risky_call_1(),
    risky_call_2(),
    return_exceptions=True  # errores se devuelven como valores, no se lanzan
)
for r in results:
    if isinstance(r, Exception):
        logger.error(f"Task failed: {r}")
```

2.3 TaskGroup (Python 3.11+) — structured concurrency

Reemplaza gather. Si una task falla, CANCELA todas las demás. No hay zombies.

```
# MARIS: cargar contexto con cancelación automática
async def load_user_context_safe(user_id: str) -> PipelineContext:
    async with asyncio.TaskGroup() as tg:
        t_profile = tg.create_task(supabase.get_profile(user_id))
        t_anchors = tg.create_task(supabase.get_anchors(user_id))
        t_history = tg.create_task(supabase.get_recent_messages(user_id,
limit=10))
        t_crisis  = tg.create_task(supabase.get_crisis_state(user_id))

    # Si cualquiera falla, TODAS se cancelan y se lanza ExceptionGroup
    return PipelineContext(
        profile=t_profile.result(),
        anchors=t_anchors.result(),
        history=t_history.result(),
        crisis_state=t_crisis.result(),
    )
```

Diferencias clave:

	`gather()`	`TaskGroup`
Error handling	Una falla no cancela las demás	Una falla cancela TODAS
Tasks dinámicas	No (lista fija)	Sí (create_task en el contexto)
Excepciones	Primera excepción o return_exceptions	ExceptionGroup (todas juntas)
Python version	3.4+	3.11+

	`gather()`	`TaskGroup`
Usar en MARIS	Para cosas que pueden fallar independientemente	Para cosas que NECESITAN todas para continuar

2.4 asyncio.create_task() — fire-and-forget

```
# MARIS: disparar background work después de enviar respuesta
async def handle_message(user_id: str, message: str):
    # Procesar y responder primero
    response = await generate_response(user_id, message)

    # Disparar tasks en background — NO esperamos resultado
    asyncio.create_task(brain_update(user_id, message, response))
    asyncio.create_task(increment_msg_count(user_id))
    asyncio.create_task(save_mode(user_id))

    return response
```

PELIGRO: Si no guardas referencia al task, el garbage collector puede matarlo:

```
# INCORRECTO — task puede ser garbage collected
asyncio.create_task(brain_update(user_id, message, response))

# CORRECTO — mantener referencia
background_tasks = set()

task = asyncio.create_task(brain_update(user_id, message, response))
background_tasks.add(task)
task.add_done_callback(background_tasks.discard)  # limpia cuando termina
```

2.5 BackgroundTasks (FastAPI) vs asyncio.create_task

```
from fastapi import BackgroundTasks

# Opción A: FastAPI BackgroundTasks
# Corre DESPUÉS de enviar la response. Secuencial por defecto.
@app.post("/chat")
async def chat(msg: ChatRequest, background_tasks: BackgroundTasks):
    response = await generate_response(msg)
    background_tasks.add_task(brain_update, msg.user_id, msg.text)
    background_tasks.add_task(save_mode, msg.user_id)
```

```
    return response

# Opción B: asyncio.create_task
# Corre INMEDIATAMENTE en paralelo. Más control.
@app.post("/chat")
async def chat(msg: ChatRequest):
    response = await generate_response(msg)
    asyncio.create_task(brain_update(msg.user_id, msg.text))
    asyncio.create_task(save_mode(msg.user_id))
    return response
```

	`BackgroundTasks`	`asyncio.create_task`
Cuándo corre	Después de enviar response	Inmediatamente
Orden	Secuencial (uno tras otro)	Paralelo
Error handling	Si falla, log y continúa	Si falla y no capturaste, exception lost
Lifecycle	Ligado al request	Vive en el event loop
MARIS:	Para tasks simples (msg_count)	Para tasks que deben correr en paralelo (brain_update + crystallize)

2.6 run_in_executor — sync code sin bloquear

```
from concurrent.futures import ThreadPoolExecutor
import asyncio

# Pool dedicado para PyTorch — limita concurrencia a 2 threads
EMBEDDING_POOL = ThreadPoolExecutor(max_workers=2, thread_name_prefix="embedding")

# MARIS: sentence-transformers embedding sin bloquear el event loop
async def compute_embedding(text: str) -> list[float]:
    loop = asyncio.get_running_loop()
    # model.encode() es sync y CPU-bound, pero PyTorch libera el GIL
    embedding = await loop.run_in_executor(
        EMBEDDING_POOL,
        lambda: model.encode(text, normalize_embeddings=True).tolist()
    )
    return embedding
```

```
# Alternativa FastAPI-native (más simple):
from starlette.concurrency import run_in_threadpool

async def compute_embedding_simple(text: str) -> list[float]:
    return await run_in_threadpool(
        lambda: model.encode(text, normalize_embeddings=True).tolist()
    )
```

Para MARIS específicamente: - `run_in_executor(EMBEDDING_POOL)` para `sentence-transformers` — max 2 workers porque cada embedding usa ~500MB RAM - `run_in_threadpool` para Anthropic sync SDK si no puedes migrar a async - NUNCA `run_in_executor` para Supabase queries — usa supabase-py async client directamente

---

3. SSE STREAMING + BACKGROUND WORK SIMULTÁNEO

Patrón actual de MARIS (threading.Thread + asyncio.Queue):

El bridge sync-to-async con threading.Thread funciona pero es frágil. Patrón mejorado:

```
from sse_starlette.sse import EventSourceResponse
import asyncio

async def stream_chat(user_id: str, message: str):
    queue: asyncio.Queue[str | None] = asyncio.Queue()

    async def producer():
        """Genera tokens y los pone en la queue."""
        try:
            # Si Claude SDK es sync, correr en executor
            loop = asyncio.get_running_loop()

            def sync_stream():
                """Corre en thread separado, pone tokens en queue thread-safe."""
                for token in claude_client.stream(message):
                    # asyncio.Queue NO es thread-safe — usar call_soon_threadsafe
                    loop.call_soon_threadsafe(queue.put_nowait, token)
                loop.call_soon_threadsafe(queue.put_nowait, None)  # señal de fin

            await loop.run_in_executor(None, sync_stream)
        except Exception as e:
            loop.call_soon_threadsafe(queue.put_nowait, None)
            logger.error(f"Stream error: {e}")
```

```
    async def event_generator():
        """Lee de la queue y yield SSE events."""
        # Disparar producer en background
        producer_task = asyncio.create_task(producer())

        full_response = []
        while True:
            token = await asyncio.wait_for(queue.get(), timeout=30.0)
            if token is None:
                break
            full_response.append(token)
            yield {"event": "token", "data": token}

        # Stream terminó — disparar background tasks
        response_text = "".join(full_response)
        asyncio.create_task(post_stream_tasks(user_id, message, response_text))
        yield {"event": "done", "data": ""}

    return EventSourceResponse(event_generator())

async def post_stream_tasks(user_id: str, message: str, response: str):
    """5 background tasks en paralelo después del stream."""
    async with asyncio.TaskGroup() as tg:
        tg.create_task(brain_update(user_id, message, response))
        tg.create_task(increment_msg_count(user_id))
        tg.create_task(save_mode(user_id))
        tg.create_task(crystallize_if_needed(user_id))
        tg.create_task(auto_sleep_check(user_id))
```

Punto clave: `asyncio.Queue` NO es thread-safe. Cuando produces desde un thread (sync SDK) y consumes desde async, debes usar `loop.call_soon_threadsafe(queue.put_nowait, item)`.

---

4. CONNECTION POOLING (Hussein Nasser)

Principio: Crear conexiones es CARO. Reutilízalas.

```
import httpx

# INCORRECTO — nueva conexión TCP + TLS handshake por cada request
async def call_claude_bad(prompt: str):
    async with httpx.AsyncClient() as client:  # nueva conexión cada vez
        return await client.post("https://api.anthropic.com/v1/messages", ...)
```

```
# CORRECTO — pool de conexiones reutilizable
class ClaudeClient:
    def __init__(self):
        self._client = httpx.AsyncClient(
            base_url="https://api.anthropic.com",
            limits=httpx.Limits(
                max_connections=20,       # max conexiones totales
                max_keepalive_connections=10,  # mantener vivas
                keepalive_expiry=30,      # segundos antes de cerrar idle
            ),
            timeout=httpx.Timeout(
                connect=5.0,    # timeout de conexión
                read=30.0,      # timeout de lectura (Claude puede tardar)
                write=5.0,
                pool=10.0,      # timeout esperando conexión del pool
            ),
            headers={"Authorization": f"Bearer {API_KEY}"},
        )

    async def generate(self, prompt: str) -> str:
        response = await self._client.post("/v1/messages", json={...})
        return response.json()

    async def close(self):
        await self._client.aclose()

# Lifecycle en FastAPI
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.claude = ClaudeClient()
    app.state.supabase = AsyncSupabaseClient()
    yield
    await app.state.claude.close()
    await app.state.supabase.close()

app = FastAPI(lifespan=lifespan)
```

Para Supabase: Mismo patrón. Un AsyncClient global con connection pooling, NO crear cliente por request.

---

5. RACE CONDITIONS Y SEGURIDAD

5.1 Por qué MARIS tiene race conditions

Los bugs del audit: `RateGuard`, `WarmthGuard`, `search._rate`, `long_term.anchors`, `CrisisState` — todos comparten estado mutable entre requests concurrentes sin protección.

```
# PELIGROSO — código actual tipo
class RateGuard:
    def __init__(self):
        self.counts = defaultdict(int)  # compartido entre TODAS las requests

    def check(self, user_id: str) -> bool:
        self.counts[user_id] += 1  # race condition entre requests concurrentes
        return self.counts[user_id] < 10
```

5.2 asyncio.Lock vs threading.Lock

```
# asyncio.Lock — para código async (coroutines)
# NUNCA bloquea el event loop, cede control mientras espera
lock = asyncio.Lock()

async def safe_increment(user_id: str):
    async with lock:  # solo una coroutine a la vez
        count = await get_count(user_id)
        await set_count(user_id, count + 1)

# threading.Lock — para código con threads reales
# BLOQUEA el thread, no usar en async context
import threading
thread_lock = threading.Lock()

def sync_safe_increment(user_id: str):
    with thread_lock:
        count = get_count_sync(user_id)
        set_count_sync(user_id, count + 1)
```

REGLA ABSOLUTA: Nunca uses `threading.Lock` en código async. Bloquea el event loop entero. Nunca uses `asyncio.Lock` en código threaded. No funciona (no hay event loop en el thread).

5.3 Patrón per-user lock (para MARIS)

```
class UserLockManager:
    """Un lock por usuario. Requests del mismo usuario se serializan.
    Requests de usuarios diferentes corren en paralelo."""

    def __init__(self):
        self._locks: dict[str, asyncio.Lock] = {}
        self._meta_lock = asyncio.Lock()  # protege el dict de locks

    async def acquire(self, user_id: str) -> asyncio.Lock:
        async with self._meta_lock:
            if user_id not in self._locks:
                self._locks[user_id] = asyncio.Lock()
            return self._locks[user_id]

# Uso en MARIS
user_locks = UserLockManager()

async def handle_message(user_id: str, message: str):
    lock = await user_locks.acquire(user_id)
    async with lock:
        # Todo el pipeline de este usuario es serial
        # Pero otros usuarios corren en paralelo
        context = await load_user_context(user_id)
        response = await generate_response(context, message)
        await save_state(user_id, context)
        return response
```

5.4 Shared Nothing — la mejor solución

```
# EN VEZ DE estado compartido en memoria:
class RateGuard:
    counts = defaultdict(int)  # PELIGRO: compartido, mutable, sin lock

# USAR Supabase/Redis como source of truth:
class RateGuard:
    def __init__(self, supabase: AsyncSupabaseClient):
        self.db = supabase

    async def check(self, user_id: str) -> bool:
        # Supabase maneja la concurrencia — cada query es atómica
        result = await self.db.rpc("increment_and_check_rate", {
            "p_user_id": user_id,
            "p_max_rate": 10,
            "p_window_seconds": 60,
```

```
        }).execute()
        return result.data["allowed"]
```

Principio (Hussein Nasser): "Si tienes estado compartido, tienes bugs. La pregunta es cuándo los vas a encontrar." Mover estado a la base de datos (Supabase) o a Redis. El proceso de FastAPI queda stateless.

5.5 Fixes específicos para los bugs del audit

```
# FIX 1: RateGuard — mover a Supabase
# Crear tabla: rate_limits (user_id, count, window_start)
# Usar RPC con operación atómica

# FIX 2: WarmthGuard — mismo patrón
# El warmth score per-user va en la sesión de Supabase, no en memoria

# FIX 3: search._rate — rate limiting en Redis o Supabase
# NUNCA defaultdict en un singleton FastAPI

# FIX 4: long_term.anchors — cargar per-request, no cachear en memoria
async def get_anchors(user_id: str) -> list[Anchor]:
    # Cada request carga fresh de Supabase
    result = await supabase.table("anchors").select("*").eq("user_id",
user_id).execute()
    return [Anchor(**row) for row in result.data]

# FIX 5: CrisisState — ya está en Supabase (resuelto en audit)
# Pero asegurar que SIEMPRE se lee de DB, nunca de variable en memoria
```

---

6. MESSAGE BUS / EVENT-DRIVEN (Cosmic Python)

6.1 El problema en MARIS

El pipeline actual tiene side effects mezclados con lógica principal: - `brain_update` se llama directamente después de generar respuesta - `crystallize` se llama inline - Si uno falla, afecta al flujo principal

6.2 Domain Events + Message Bus

```
from dataclasses import dataclass, field
from typing import Callable, Type

# --- Events (hechos que ocurrieron) ---
@dataclass(frozen=True)
class MessageProcessed:
    user_id: str
    message: str
    response: str
    session_id: str

@dataclass(frozen=True)
class MoodChanged:
    user_id: str
    old_mood: float
    new_mood: float

@dataclass(frozen=True)
class CrisisDetected:
    user_id: str
    level: str
    message: str

# --- Commands (intenciones de hacer algo) ---
@dataclass(frozen=True)
class UpdateBrain:
    user_id: str
    message: str
    response: str

@dataclass(frozen=True)
class CheckCrystallization:
    user_id: str

# --- Message Bus ---
class MessageBus:
    def __init__(self):
        self._handlers: dict[Type, list[Callable]] = {}

    def register(self, event_type: Type, handler: Callable):
        self._handlers.setdefault(event_type, []).append(handler)

    async def handle(self, event):
        handlers = self._handlers.get(type(event), [])
        for handler in handlers:
            try:
                await handler(event)
            except Exception as e:
```

```
                logger.error(f"Handler {handler.__name__} failed for
{type(event).__name__}: {e}")
                # Events: log and continue (otros handlers deben correr)
                # Commands: re-raise (el caller necesita saber)

# --- Wiring (composition root) ---
bus = MessageBus()
bus.register(MessageProcessed, handle_brain_update)
bus.register(MessageProcessed, handle_msg_count)
bus.register(MessageProcessed, handle_save_mode)
bus.register(MessageProcessed, handle_crystallize)
bus.register(MessageProcessed, handle_auto_sleep)
bus.register(CrisisDetected, handle_crisis_escalation)
bus.register(MoodChanged, handle_mood_notification)

# --- En el pipeline ---
async def process_message(user_id: str, message: str) -> str:
    response = await generate_response(user_id, message)

    # Emitir evento — el bus se encarga de los side effects
    await bus.handle(MessageProcessed(
        user_id=user_id,
        message=message,
        response=response,
        session_id=current_session_id,
    ))

    return response
```

Ventaja: El pipeline principal NO sabe qué side effects existen. Agregar uno nuevo = registrar un handler. Eliminar uno = quitar el registro. Testing = bus con 0 handlers.

6.3 Command vs Event

	Command	Event
Nombre	Imperativo: `UpdateBrain`	Pasado: `MessageProcessed`
Handlers	Exactamente 1	0 o más
Si falla	Re-raise, el caller decide	Log and continue
Ejemplo MARIS	`SendCrisisAlert`	`CrisisDetected`

6.4 Unit of Work (Cosmic Python)

```
from contextlib import asynccontextmanager

class AsyncUnitOfWork:
    """Transacción atómica: todo se guarda o nada."""

    def __init__(self, supabase: AsyncSupabaseClient):
        self._supabase = supabase
        self._events: list = []  # eventos pendientes

    @asynccontextmanager
    async def __call__(self):
        try:
            yield self
            # Commit: guardar todo
            await self._supabase.rpc("commit_transaction").execute()
            # Solo emitir eventos si el commit fue exitoso
            for event in self._events:
                await bus.handle(event)
        except Exception:
            await self._supabase.rpc("rollback_transaction").execute()
            raise
        finally:
            self._events.clear()

    def add_event(self, event):
        self._events.append(event)

# Uso
async def process_and_save(user_id: str, message: str):
    uow = AsyncUnitOfWork(supabase)
    async with uow():
        response = await generate_response(user_id, message)
        await uow.save_message(user_id, message, "user")
        await uow.save_message(user_id, response, "assistant")
        uow.add_event(MessageProcessed(user_id=user_id, message=message,
response=response))
        # Si save_message falla, se hace rollback Y no se emiten eventos
```

---

7. TASK QUEUE: CUÁNDO GRADUAR

Escalera de complejidad:

```
Nivel 1: BackgroundTasks (FastAPI built-in)
    ✓ Simple, in-process
    ✗ Se pierde si el proceso muere
    ✗ No retry, no monitoring
    → MARIS HOY: suficiente para brain_update, msg_count

Nivel 2: asyncio.create_task + TaskGroup
    ✓ Paralelo, structured cancellation
    ✗ Se pierde si el proceso muere
    → MARIS PRONTO: para paralelizar 5 background tasks

Nivel 3: asyncio.Queue (producer-consumer)
    ✓ Backpressure, desacopla producción de consumo
    ✗ In-process, se pierde al morir
    → MARIS: para SSE streaming (ya lo usas)

Nivel 4: ARQ (Async Redis Queue)
    ✓ Persiste en Redis, retry automático, async-native
    ✓ Ligero, diseñado para FastAPI
    ✗ Necesita Redis
    → MARIS FUTURO: cuando tengas >100 users concurrentes

Nivel 5: Celery + Redis/RabbitMQ
    ✓ Battle-tested, monitoring (Flower), scheduling
    ✗ Pesado, no async-native, overhead de setup
    → MARIS: NO necesario. ARQ es mejor para tu caso.
```

ARQ ejemplo (para cuando MARIS lo necesite):

```
# worker.py
from arq import create_pool
from arq.connections import RedisSettings

async def brain_update(ctx, user_id: str, message: str, response: str):
    """Corre en worker separado, persiste en Redis."""
    await update_brain_vectors(user_id, message, response)

class WorkerSettings:
    functions = [brain_update]
    redis_settings = RedisSettings(host="redis-host")

# api.py — enqueue desde FastAPI
redis = await create_pool(RedisSettings(host="redis-host"))
await redis.enqueue_job("brain_update", user_id, message, response)
```

8. CIRCUIT BREAKER PARA APIs EXTERNAS

Para Claude API (2-5s response time, puede fallar):

```
import asyncio
from enum import StrEnum
from time import monotonic

class CircuitState(StrEnum):
    CLOSED = "closed"        # funcionando normal
    OPEN = "open"            # rechazando requests
    HALF_OPEN = "half_open"  # probando con 1 request

class AsyncCircuitBreaker:
    def __init__(
        self,
        name: str,
        fail_threshold: int = 5,
        reset_timeout: float = 60.0,
        half_open_max: int = 1,
    ):
        self.name = name
        self.fail_threshold = fail_threshold
        self.reset_timeout = reset_timeout
        self.half_open_max = half_open_max

        self._state = CircuitState.CLOSED
        self._fail_count = 0
        self._last_failure_time = 0.0
        self._half_open_count = 0
        self._lock = asyncio.Lock()

    async def call(self, func, *args, **kwargs):
        async with self._lock:
            if self._state == CircuitState.OPEN:
                if monotonic() - self._last_failure_time > self.reset_timeout:
                    self._state = CircuitState.HALF_OPEN
                    self._half_open_count = 0
                else:
                    raise CircuitBreakerOpen(f"{self.name} circuit is OPEN")

            if self._state == CircuitState.HALF_OPEN:
                if self._half_open_count >= self.half_open_max:
                    raise CircuitBreakerOpen(f"{self.name} circuit is HALF_OPEN,
max trials reached")
                self._half_open_count += 1
```

```
        try:
            result = await func(*args, **kwargs)
            async with self._lock:
                self._fail_count = 0
                self._state = CircuitState.CLOSED
            return result
        except Exception as e:
            async with self._lock:
                self._fail_count += 1
                self._last_failure_time = monotonic()
                if self._fail_count >= self.fail_threshold:
                    self._state = CircuitState.OPEN
                    logger.warning(f"Circuit {self.name} OPENED after
{self._fail_count} failures")
            raise

class CircuitBreakerOpen(Exception):
    pass

# Uso en MARIS
claude_breaker = AsyncCircuitBreaker("claude_api", fail_threshold=3,
reset_timeout=30.0)

async def generate_response_safe(user_id: str, message: str) -> str:
    try:
        return await claude_breaker.call(call_claude_api, user_id, message)
    except CircuitBreakerOpen:
        # Fallback: respuesta genérica o cached
        return "Estoy teniendo problemas técnicos. ¿Podemos intentar de nuevo en
un momento?"
```

---

9. TIMEOUT PATTERNS

```
# Timeout para Claude API (puede colgar)
async def call_claude_with_timeout(prompt: str, timeout: float = 15.0) -> str:
    try:
        return await asyncio.wait_for(
            call_claude_api(prompt),
            timeout=timeout,
        )
    except asyncio.TimeoutError:
        logger.warning(f"Claude API timeout after {timeout}s")
        raise
```

```
# Timeout con fallback
async def generate_response(user_id: str, message: str) -> str:
    try:
        return await asyncio.wait_for(call_claude_api(message), timeout=15.0)
    except (asyncio.TimeoutError, CircuitBreakerOpen):
        # Fallback 1: retry con prompt más corto
        try:
            short_prompt = truncate_context(message)
            return await asyncio.wait_for(call_claude_api(short_prompt),
timeout=10.0)
        except (asyncio.TimeoutError, CircuitBreakerOpen):
            # Fallback 2: respuesta empática genérica
            return get_fallback_response(user_id)
```

---

10. PATRÓN COMPLETO PARA MARIS

Arquitectura target: pipeline async con background tasks

```
# main.py — composition root
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: inicializar recursos compartidos
    app.state.claude = ClaudeClient()           # connection pool
    app.state.supabase = AsyncSupabaseClient()  # connection pool
    app.state.embedding_pool = ThreadPoolExecutor(max_workers=2)
    app.state.bus = create_message_bus()         # wired handlers
    app.state.claude_breaker = AsyncCircuitBreaker("claude", fail_threshold=3)
    app.state.user_locks = UserLockManager()

    yield

    # Shutdown: limpiar
    await app.state.claude.close()
    await app.state.supabase.close()
    app.state.embedding_pool.shutdown(wait=True)

app = FastAPI(lifespan=lifespan)

# --- Endpoint SSE ---
@app.post("/v1/chat/stream")
```

```
async def chat_stream(request: ChatRequest):
    user_id = request.user_id
    message = request.message

    # Per-user lock: serializar mensajes del mismo usuario
    lock = await app.state.user_locks.acquire(user_id)

    async def event_generator():
        async with lock:
            # 1. Cargar contexto (paralelo)
            context = await load_user_context(user_id, app.state.supabase)

            # 2. Safety check (sync PyTorch via executor)
            loop = asyncio.get_running_loop()
            embedding = await loop.run_in_executor(
                app.state.embedding_pool,
                lambda: model.encode(message).tolist()
            )

            # 3. Stream response (Claude con circuit breaker)
            queue: asyncio.Queue = asyncio.Queue()
            full_response = []

            async def produce():
                try:
                    async for token in await app.state.claude_breaker.call(
                        stream_claude, context, message
                    ):
                        await queue.put(token)
                        full_response.append(token)
                except Exception as e:
                    logger.error(f"Stream error: {e}")
                finally:
                    await queue.put(None)

            producer = asyncio.create_task(produce())

            while True:
                token = await asyncio.wait_for(queue.get(), timeout=30.0)
                if token is None:
                    break
                yield {"event": "token", "data": token}

            yield {"event": "done", "data": ""}

        # 4. Background tasks (FUERA del lock — otros mensajes pueden entrar)
        response_text = "".join(full_response)
        asyncio.create_task(
            fire_post_message_events(app.state.bus, user_id, message,
```

```
response_text)
        )

    return EventSourceResponse(event_generator())

async def fire_post_message_events(bus: MessageBus, user_id: str, message: str,
response: str):
    """Emitir evento — el bus dispara los 5 handlers en paralelo."""
    await bus.handle(MessageProcessed(
        user_id=user_id,
        message=message,
        response=response,
        session_id="current",
    ))
```

RESUMEN: DECISIONES PARA MARIS

Problema actual	Solución	Prioridad
Claude sync SDK bloquea event loop	`run_in_executor` + `asyncio.Queue` bridge	ALTA
5 background tasks secuenciales	`asyncio.TaskGroup` en paralelo	ALTA
Race conditions (RateGuard, etc.)	Mover estado a Supabase, `asyncio.Lock` per-user	ALTA
Sin connection pooling	`httpx.AsyncClient` global con limits	MEDIA
Sin timeout en Claude API	`asyncio.wait_for` + circuit breaker	MEDIA
Side effects acoplados al pipeline	Message Bus + Domain Events	MEDIA
No retry en background tasks	ARQ (cuando >100 users)	BAJA
Sin transacciones atómicas	Unit of Work con Supabase	BAJA

Sources: - [Real Python: Speed Up Your Python Program With Concurrency] - [Real Python: Asyncio Hands-On Walkthrough] - [Cosmic Python: Unit of Work Pattern] - [Cosmic Python: Events]

and the Message Bus - Cosmic Python: Going to Town on the Message Bus - FastAPI: Background Tasks - FastAPI: Concurrency and async/await - BackgroundTasks vs Threads vs Async (Medium) - Better Stack: Background Tasks in FastAPI - Why TaskGroup and Timeout Are Crucial in Python 3.11 Asyncio - Sentry: run_in_executor vs run_in_threadpool - Maximizing PyTorch Throughput with FastAPI - FastAPI Race Conditions (DataSci Ocean) - SuperFastPython: Asyncio Race Conditions - Python Dictionary Async Safety (Medium) - Hussein Nasser: Threads and Connections in Backend Applications - Python Asyncio Sync Primitives (docs) - Python Coroutines and Tasks (docs) - PyBreaker Circuit Breaker (GitHub) - aiobreaker for async circuit breaking - ARQ Async Redis Queue - Celery vs ARQ Comparison (Leapcell) - OneUptime: Circuit Breakers in Python - Using asyncio Queues with SSE (Medium) - Inngest: Python asyncio shared state pitfalls

AUDITORIA DE CODIGO

AUDITORÍA COMPLETA DE CÓDIGO — MARIS

Backend (9,550 LOC Python) + iOS (7,071 LOC Swift)

Fecha: Abril 2026

RESUMEN EJECUTIVO

Métrica	Backend	iOS
Archivos auditados	28	22
LOC total	9,550	7,071
God objects	2 (Engine, WarmthGuard)	3 (ChatViewModel, SessionManager, DashboardView)

Métrica	Backend	iOS
Tests	0 automatizados	0
Analytics events	0	0
Protocols/DI	0	0
Thread safety gaps	5 módulos	3 módulos
Pasos por mensaje	51	N/A
Monetización	N/A	DESHABILITADA

BACKEND: FLOW DE UN MENSAJE (51 pasos)

```
POST /chat → Main.py
  1-7: fast_check (x2 DUPLICADO), auth, rate limit, sanitize, repetition
    ├── EXIT: lexical crisis
    ├── EXIT: repetition
    └── CONTINUE

→ understand.py :: listen() [388 LOC, 1 FUNCIÓN, 40 if/else]
  8-30: warmth → friction → memory → search → embeddings
        → crisis_classifier → lethal_means → frontal
        → psychosis → mania → selector → temporal
        → eip.evaluate → physics → circadian → warmth
        → indirect_signals → resolve_state → positive
        → crisis_gradient → build_prompt → prepare_msgs
  OUTPUT: Understanding (30+ campos)

→ respond.py :: respond() [175 LOC]
  31-42: Route LLM → stream → 5 post-processors → VETO gate
         → identity → sovereignty → clean → spanish
  43: yield SSE text
  44-51: friction, deformation, temporal, insight, takeaway,
         background tasks (brain, msg_count, save_mode, anchor)
```

BACKEND: SCORES POR ARCHIVO

Archivo	LOC	Read	Test	Mod	Simp
models.py	194	5	5	5	5
engine.py	231	4	1	1	3
Main.py	243	3	2	2	2
respond.py	277	3	2	3	2
understand.py	498	3	1	2	1

BACKEND: SCORES POR MÓDULO (top issues)

Módulo	LOC	Issue
eip.py	961	Más grande. 250 líneas datos inline
warmth.py	569	GOD OBJECT: 9 detectores en 1 clase
clinical_detectors	548	Duplica patrón de safety_detectors
safety_detectors	520	_build_semantic_index copy-pasteado
Storage.py	347	GOD OBJECT: auth+analytics+memory+session
temporal.py	216	MEJOR DISEÑADO: state separado de logic

BACKEND: 7 HALLAZGOS CRÍTICOS

1. **Engine = God Object** — 22 imports, 20+ atributos. Nada testeable sin Engine completo
2. **listen() = 388 líneas** — 1 función, 40 if/else, magic numbers
3. **Duplicación masiva en detectores** — 11 detectores con el mismo patrón copy-pasteado
4. **~800 líneas datos inline** — strings Python que deberían ser JSON externo
5. **Thread safety gaps** — 5 módulos con estado mutable sin locks
6. **/api/chat = pipeline paralelo** — duplica lógica con diferencias sutiles

7. **Bounded context violations** — session.py importa de los 3 contextos

---

iOS: USER FLOW COMPLETO

```
App Launch → ContentView gates:
  ├── No consent → ConsentView ("Boe: una presencia que escucha")
  ├── No onboarding → EmotionalOnboarding (feeling→need→"Habla")
  └── Ready → MainTabView
        ├── Tab 0: DashboardView (greeting, plans, night review, actions)
        └── Tab 1: ChatView (loadGreeting→send→SSE→done→persist)
              ├── Menu → Ayuda, Notas, Sobre ti, Info, Ajustes
              └── Premium → Packs $4.99/$14.99 (DESHABILITADO)
```

iOS: SCORES POR ARCHIVO

Archivo	LOC	Read	Test	Mod
MainTabView	31	5	4	5
ContentView	17	5	3	4
ConsentView	113	4	3	4
EmotionalOnboarding	150	4	3	4
StoreManager	79	4	3	4
NotificationService	220	4	2	3
LocalVault	415	3	2	3
PhysicsEngine	378	3	1	2
ChatViewModel	**459**	**1**	**1**	**1**
DashboardView	**578**	**2**	**1**	**1**
SessionManager	**150**	**2**	**2**	**2**
APIClient	108	2	1	2

Archivo	LOC	Read	Test	Mod
MessageBubble	395	2	1	1
SettingsView	471	2	1	1

iOS: HALLAZGOS CRÍTICOS

1. ANALYTICS = CERO

No existe un solo evento trackeado. Cero frameworks importados. No Mixpanel, Firebase, Amplitude. Volando 100% ciego.

Eventos que DEBERÍAN existir: ~50+ categorizados en onboarding (7), engagement (6), safety (7), features (11), dashboard (10), monetización (7), retención (2).

2. ONBOARDING FALLA EL GRUNT TEST

- ConsentView: "Una presencia que escucha" — NO dice qué es la app, qué hace, ni por qué importa
- EmotionalOnboarding: calibra emoción pero no explica QUÉ hará Boe con esa información
- Valor hasta mensaje 1: requiere consent + onboarding + navegar a chat + escribir + esperar API = **5 acciones mínimo antes del primer valor**
- No hay: explicación del producto, ejemplo de conversación, social proof, ni credibilidad

3. MONETIZACIÓN DESHABILITADA

- `hasMessages` retorna `true` siempre (línea 122: "Limits disabled during development")
- El paywall nunca se activa
- Precios hardcodeados en dólares (no usa product.displayPrice)
- Sin validación server-side de compras
- Si el usuario reinstala, pierde mensajes comprados

4. ChatViewModel = GOD OBJECT (459 LOC)

16 responsabilidades: mensajes, greeting, name detection, intent routing, 12 comandos locales, muletillas, semantic cache, message limits, API, SSE processing, done-event side effects, error

handling, conversion timing, account banner, connection status, persistence. 8 singletons directos. `send()` = 275 líneas con 6+ niveles de nesting.

5. 11+ SINGLETONS (.shared)

SessionManager, APIClient, LocalVault, StoreManager, PhysicsEngine, SemanticCache, NotificationService, IntentRouter, ReminderService, VoiceOutput, WeatherService, CalendarService, HealthKitService. Zero dependency injection. Zero protocols. Imposible de testear.

6. CONCURRENCY ISSUES

- PhysicsEngine: `@MainActor` parcial, `saveToDisk` captura self strongly en non-Sendable
- SemanticCache: read fuera de queue, write dentro — race condition
- LocalVault: `queue.sync` desde MainActor callers — potencial deadlock

---

DEPENDENCIAS CRUZADAS (BACKEND)

```
Engine (GOD) —imports→ 22 modules
  ├── CONVERSATION: identity, llm, Integrity, warmth, calibrator, Processor, Brain
  ├── CLINICAL: eip, clinical_detectors, safety_detectors, crisis_classifier,
audit, auth
  ├── FLOW: intent, selector, search
  └── MEMORY: Storage, _db, long_term, dream, temporal, summarizer, session

Boundary violations:
  - eip.py (Safety) → Processor.detect_lang (Conversation)
  - session.py (Memory) → selector + temporal + eip (3 contextos)
  - dream.py (Memory) → Brain._lock privado (Conversation)
  - audit.py (Clinical) → _db (Memory)
  - clinical_detectors → safety_detectors._is_spanish (cross-file private)
```

LO QUE ESTÁ BIEN (no todo es malo)

Backend fortalezas:

- **temporal.py** — Mejor diseñado: state (dataclass) separado de logic (monitor). Modelo a seguir.
- **models.py** — Limpio, hoja sin dependencias, bien tipado
- **identity.py** — Puro, sin estado, testeable
- **summarizer.py** — Limpio, recibe dependencias por constructor
- **search.py** — Pequeño, enfocado, bien nombrado
- **Pipeline de safety es REAL** — 248 vectores, 15 clusters semánticos, C-SSRS grade. Esto es genuinamente difícil de replicar.
- **Physics brain es único** — Momentum, energy, weight, load con derivadas. No existe en ningún competidor.

iOS fortalezas:

- **MainTabView** — Simple, limpio, hace una cosa
- **ContentView** — Gate logic clara y mínima
- **ConsentView** — Flujo funcional con crisis resources siempre accesibles
- **EmotionalOnboarding** — Rápido (3 taps), no invasivo
- **SemanticCache** — Concepto correcto (NLEmbedding local para ahorrar API calls)
- **La app FUNCIONA en el iPhone** — Para un solo dev, tener 39 archivos Swift funcionando con SSE streaming, HealthKit, widget, y onboarding es significativo

RECURSOS

Canales Técnicos de Referencia

Para arquitectura, code reading, y refactoring del backend de MARIS

Code Reading & Walkthroughs

- **The Primeagen** — Code reviews en vivo de repos open source, critica arquitecturas, navegación eficiente
- **George Hotz (Archive)** — Sessions de cómo entrar a código desconocido y dominarlo en horas
- **Code Aesthetics** — Legibilidad: por qué ciertos patrones son más fáciles de escanear

Arquitectura y Diseño de Sistemas

- **Codely (español)** — Arquitectura Hexagonal, DDD, SOLID. Referente en español para integrar AI en sistemas modulares

- **ByteByteGo (Alex Xu)** — System Design con diagramas claros. Arquitecturas de empresas grandes (TikTok, WhatsApp)
- **CodeOpinion (Derek Compo)** — Debate Monolitos Modulares vs Microservicios. Cuándo separar y cuándo no
- **Hussein Nasser** — Lo que pasa bajo el capó: protocolos, bases de datos, proxies. Optimizar comunicación entre servicios

Python & Refactoring

- **ArjanCodes** — Python: transformar espagueti en arquitectura modular y testeable. Patrones de diseño aplicados
- **James Shore** — Desarrollo evolutivo: la arquitectura debe crecer CON el código, no antes

Bajo Nivel

- **Low Level Learning** — Memoria, procesos, rendimiento. Perspectiva sólida sobre por qué las cosas son lentas o rápidas

---

Aplicación a MARIS Backend

- **ArjanCodes** para refactorizar el pipeline de 23 pasos (módulos más testeables)
- **Codely** para evaluar si DDD aplica a la separación de dominios (crisis, scroll, plans, physics)
- **ByteByteGo** para el diseño de la arquitectura cuando MARIS escale (SSE, caching, Supabase, queue)
- **Hussein Nasser** para optimizar la comunicación Railway↔Supabase↔Claude API